

claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf_a2b97cff-dcd\agent-aa25b6dc407e098c3.jsonl`  
- SHA-256: `3f47a44478a67447cf7f3aa94c6b198e61df8041d37e6cd28a1eabc1c1725a46`  
- Source modified: `2026-06-21T14:08:20+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_a2b97cff-dcd`  
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`
```

Transcript

1. user (2026-06-21T14:05:19.926Z)

You are a completeness critic for a plan to ship the engine as a downloadable batteries-included Python app. Given the digest below, find: GAPS (topics not covered that the plan needs), MISSING_TOPICS (e.g. data migration/upgrade, uninstall, telemetry/observability for the app, secrets handling for the Gemini key, Windows-specific path/CRLF/long-path issues, auto-update, first-run wizard, how training is gated/long-running, how the SPA versions stay in sync with the engine), UNVERIFIED_OR_REFUTED claims that weaken a recommendation, CONTRADICTIONS between topics, and concrete FOLLOWUPS. Be specific and harsh.

DIGEST:

```
{  
  "understanding": [  
    {  
      "area": "App launch + current packaging surface (how a user goes from nothing to a running server+worker; what `pip install .` provides; entrypoints; config/DB/output resolution; server-vs-worker process model)",  
      "summary": "Today the engine is NOT a downloadable batteries-included app ? it is a source checkout you `pip install -e .` from. The PEP 621 build (hatchling, `pyproject.toml:12-93`) installs the `backend` + `training` namespace packages and registers ONE console script, `indexer = backend.main:main` (`pyproject.toml:80-82`), but deliberately omits torch/torchvision (their CUDA/CPU wheels need `--index-url`, which PEP 621 can't express), so `pip install .` alone yields a torch-less install that cannot run the GPU/yolo_hybrid/trajectory paths. ffmpeg/ffprobe are unmanaged system deps. The intended cold-start path is: (1) install ffmpeg, (2) `pip install -e .` (or `.[dev]`), (3) install torch out-of-band, (4) run `scripts/doctor.py` / `indexer --setup` to GPU-probe + write a default `config.json` + `pip install -r requirements.txt` --user, (5) set `GEMINI_API_KEY` in a `.env`, (6) launch. The launcher resolves config.json, .env, the SQLite DB, and output/ all CWD-RELATIVE ? there is no app-data dir, no XDG/AppData home, no env override for their location ? so the running directory IS the implicit state root. Critically, server and worker are NOT separate processes for the normal serving path: the FastAPI server runs an IN-PROCESS ThreadPoolExecutor job worker (`backend.api.job_worker`), so `python -m backend.main --server` is fully self-contained. `python -m backend.worker {infer|train}` is a SEPARATE, storage-aware (pull?run?push) orchestrator built for the decoupled-compute / Cloud Run path, not a daemon the server talks to. `main()` is overloaded: `--server`, single-shot `--video-path`, `--setup`, `--debug-clip`, `--trim-*` all dispatch from the one entrypoint, defaulting to server mode when no video path is given (`backend/main.py:557-736`).",  
      "implications": [  
        "A batteries-included app cannot ship as a plain wheel: torch/torchvision are out-of-band by design (pyproject.toml:8-11,63-70). The packaging layer must provide a post-install/first-run torch bootstrap (the logic already exists in scripts/doctor.py:15-42 ? GPU-probe -> pick cu124/cpu index) or ship a platform-pinned bundle. This is THE first hard problem and is independent of the WASB conda-env split.",  
        "ffmpeg/ffprobe are unmanaged system deps checked only by doctor.py (scripts/doctor.py:84-109). A downloadable app must bundle static ffmpeg/ffprobe binaries (license-clean: ffmpeg LGPL/GPL ? verify which build) or vendor a pip-installable static-ffmpeg,
```

and point the engine at them, because today everything assumes they are on PATH.",

"ALL mutable state (config.json, .env, SQLite DB, output/, the static UI mount) is CWD-relative with NO app-home/XDG/AppData concept (backend/config/loader.py:30; backend/main.py:64-66,636-640). A packaged app run from an arbitrary launch dir (e.g. an OS app menu) would scatter or fail to find state. Packaging must introduce a resolved app-data root (config + DB + output + cache) and thread it through OutputConfig.output_dir, DEFAULT_CONFIG_PATH, load_dotenv path, and the static mount.",

"The single console entrypoint `indexer` already gives a clean app launch surface (pyproject.toml:80-82). A packaged launcher can default to `indexer --server` and rely on the in-process job worker (backend/main.py:729-736) ? NO separate worker process needs to be supervised for local serving. The separate `python -m backend.worker` is only needed for the cloud/decoupled-compute path, so a truly-local bundle can omit worker supervision entirely.",

"doctor.py uses `pip install --user -r requirements.txt` at runtime (scripts/doctor.py:23) ? a from-source assumption. A no-clone downloadable app must NOT shell out to pip against a requirements.txt that won't exist in a frozen/bundled distribution; the dependency install must move into the package build/first-run, and requirements.txt's torch lines reconciled with pyproject's deliberate omission.",

"The hardcoded 'backend/api/static' mount (backend/main.py:64-66) and the engine's own legacy dashboard exist independently of the khelsutra Vite/React SPA. Bundling the real UI means either (a) building khelsutra/web and copying dist into a package-data dir resolved via importlib.resources (not CWD), or (b) keeping the legacy static UI. Either way the CWD-relative mount must become package-relative.",

"google-genai is a HARD dependency (pyproject.toml:35) even though local-only profiles (truly-local preset sets skip_ai_handoff=true, backend/config/presets.py:44-50) never call it. For a license-clean, cloud-optional download this is fine to keep but means the binary always carries the Gemini SDK; confirm google-genai license is Apache-2.0/MIT/BSD per the guardrail.",

"A deployment.profile=truly-local preset already bundles the zero-cloud knobs (skip_ai_handoff + local storage + local_gpu) ? the packaged 'local' app can default to this profile via DEPLOYMENT_PROFILE env or a shipped config.json, giving a coherent offline default without new code (backend/config/presets.py:43-63; loader resolves env-over-file at loader.py:56-71)."

],

"risks": [

"CWD-dependence is a latent footgun: launching the packaged app from the wrong directory silently creates a fresh empty config.json/DB/output rather than erroring (loader.py:115-122 degrades to defaults; main.py:637 makedirs output). Users could lose/duplicate their index across launches.",

"doctor.py writes config.json to REPO_ROOT (scripts/doctor.py:57) while the running server reads Path('config.json') from CWD (loader.py:30) ? if the packaged launcher's CWD != the doctor's REPO_ROOT, `indexer --setup` and the server disagree on the authoritative config. This split must be unified before packaging.",

"requirements.txt and pyproject.toml have already drifted (requirements.txt:9-10 lists torch>=2.12.0/torchvision; pyproject deliberately omits them). doctor.py installs from requirements.txt, pip install . from pyproject ? two dependency sources of truth that a packaging pass must reconcile or one will silently win.",

"The WASB native shuttle model needs a SEPARATE conda Python env (torch 1.11+cu113) vs the engine's torch 2.6+cu124, and weights live in GCS (per orchestrator notes; config defaults at backend/config/models.py:61-65 point at ~/models/WASB-SBDT and a WSL conda env). This dual-env + remote-weights reality is fundamentally incompatible with a single self-contained Python bundle for the wasb_hybrid path ? a packaging plan must either gate wasb behind an optional installer step or fall back to motion/yolo_hybrid for the batteries-included default.",

"The in-process job executor means a server crash loses all queued/running jobs (swept to 'failed' on next start, main.py:642-646). For a desktop-grade local app users may expect jobs to survive a restart; this is acceptable today but worth flagging as a UX expectation gap."

],

"open_questions": [

"Should the packaged app standardize an app-data home (e.g. %APPDATA%/Khelsutra or ~/.khelsutra) and migrate config.json/.env/DB/output/cache there, and does any existing doc (khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md) already specify this? I did not read that doc in this pass.",

"Is the batteries-included default meant to be the `truly-local` profile (skip_ai_handoff, motion/yolo_hybrid, no Gemini key required) or must the bundle ship Gemini-capable out of the box (requiring the user to provide GEMINI_API_KEY)? The README still ships config.json defaulting default_segmenter to gemini (README.md:103,171), which contradicts a zero-config local-first download.",

"How should torch be delivered in a no-clone download ? bundled platform-pinned wheels, a guided first-run installer reusing doctor.py's GPU-probe logic, or a CPU-only baseline with optional GPU upgrade? This drives whether one universal artifact or per-platform artifacts are shipped."

"Should `python -m backend.worker` be exposed as a second console script (e.g. `indexer-worker`) for the cloud-serving packaging path, or remain module-invocation-only since local serving needs no separate worker?",

"Which UI ships in the local bundle ? the legacy in-engine backend/api/static dashboard or a built khelsutra/web SPA copied into package data ? and who owns the cross-repo build step?",

"Is the ffmpeg/ffprobe binary to be bundled (and which license-clean static build) or left as a documented prerequisite with a guided installer, given the project's MIT/Apache-2.0/BSD-only guardrail (ffmpeg is LGPL/GPL)?"

```
]
},
{
  "area": "Config system + deployment profiles + the decoupled-compute seam
(backend/config/: models.py, presets.py, startup.py, loader.py; the compute seam in
backend/compute/)",
  "summary": "The engine has a single typed Pydantic config
(backend/config/models.py::AppConfig) loaded by backend/config/loader.py::load_config with
strict precedence: built-in model defaults < profile preset < config.json < env
(DEPLOYMENT_PROFILE, RALLY_DETECTOR_IMPL, WASB_*) < worker `--set k=v`. A `deployment` section
(profile + compute_target) is the ONE switch that selects a coherent runtime: choosing a profile
applies a preset BUNDLE (presets.py::PROFILE_PRESETS) over existing knobs ? truly-local sets
compute_target=local_gpu / skip_ai_handoff=True / storage.backend=local; cloud-serving sets
compute_target=cloud_run / detector_impl=native / skip_ai_handoff=False / storage.backend=gcs;
cpu-mock sets compute_target=cpu_mock / detector_impl=stub / storage.backend=local. The system
is deliberately DEFAULT-OFF: with profile="" the loader is byte-identical to pre-M1 behaviour
(the shallow {**defaults, **file_data} merge), and the per-profile startup checks (startup.py)
are a strict no-op. The ONLY thing compute_target actually changes at runtime today is the
compute seam: backend/compute/__init__.py::get_compute_backend returns a CloudRunCompute
dispatcher ONLY for compute_target=="cloud_run", else None which is the unchanged in-process
path. So a SINGLE build runs both local and cloud purely by config: the same `python -m
backend.worker infer` / `backend.main --server` binary; the profile decides whether the job runs
in-process on a local GPU or is dispatched to a Cloud Run Job. Auto-detect only SUGGESTS a
profile (D15), never silently selecting cloud (= spend). Startup checks fail-fast (hard ERROR)
only on the one certain incoherence (cloud-serving with a non-cloud storage backend); everything
else (GPU/creds mismatch) is a WARNING because the detector subprocess healthcheck is the real
authority.",
  "implications": [
    "ONE build is genuinely enough for local AND cloud: a packaged app ships a single
binary/entrypoint and the user flips between truly-local and cloud-serving by setting
deployment.profile (config.json) or DEPLOYMENT_PROFILE (env). This realizes D13 'switch anytime'
with zero rebuild -- the packager should surface a profile toggle, not separate downloads.",
    "First-run UX seam already exists: save_default_config() writes a defaulted config.json.
A packaged-app onboarding should call this (or a --setup path) to materialize config.json, then
write deployment.profile after a guided choice. suggest_profile gives a SUGGESTED default to
pre-select, but D15 requires the user to confirm; the wizard must show 'we detected a GPU ->
truly-local; confirm or change', never silently pick cloud.",
    "Expose to a NON-TECH user (small, safe set): (1) profile as 3 plain-language cards
(This machine, offline & private / Our cloud / Test mode); (2) input source = local folder vs
Google Drive vs bucket (storage.input_backend/input_root); (3) output folder
(output.output_dir); (4) Gemini API key (via env) only when a profile needs the handoff; (5)
watermark text on/off (stitching.watermark). Everything else preset-driven.",
    "HIDE from a non-tech user (expert knobs): detector_impl, skip_ai_handoff, all
indexing.* windowing tuning (max_window_duration, R4/R5, velocity thresholds), wasb.*
WSL/conda/device internals, chunk_duration/overlap, ai_* handoff knobs, billing.*,
security.edge_auth/cf_* and Cloud Run env coordinates. The profile preset sets these coherently;
ship a raw-config.json 'advanced' escape hatch outside the default flow.",
    "The startup-check layer is a ready-made packaged-app health gate: on launch run
enforce_startup_checks and surface results -- a hard error (cloud-serving without a reachable
bucket) blocks the job with a clear message; warnings (no GPU visible, creds unconfirmed) become
non-blocking banners. Exactly the 'loud before any job' UX a non-tech user needs.",
    "compute_target==cloud_run is the ONLY runtime divergence today and needs
```

```

CLOUD_RUN_JOB/REGION/GOOGLE_CLOUD_PROJECT env + GCS creds the engine does NOT manage. A
'batteries-included' cloud profile in a downloadable app is therefore NOT self-contained: it
requires the user's own GCP project + a deployed Cloud Run worker image + ADC. Truly-local is
the only fully self-contained path; treat cloud-serving as 'connect to a hosted service / your
own GCP', not 'works out of the box'.",
    "cpu-mock is a gift for the installer/smoke test: runs the whole DETECT->WINDOW->STITCH
pipeline with the stub detector (no GPU/WSL, no API key), byte-deterministic. A packaged app can
run a cpu-mock self-test on first launch to prove the install works before the user provisions a
GPU or Gemini key.",
    "No credentials live in config (by design): the packaged app must source the Gemini key,
GCS ADC, and Cloud Run coordinates from env/OS credential stores, not the bundled config.json.
The packager needs a credential-entry UX that writes env, not config fields."
  ],
  "risks": [
    "Biggest mismatch with 'batteries-included downloadable app': truly-local still depends
on heavy, non-bundled externals -- torch (deliberately not a dependency; needs --index-url CUDA
wheels), the SEPARATE WASB conda env (torch 1.11+cu113) reached via WSL on Windows, system
ffmpeg/ffprobe, and WASB weights pinned to a ~/models path / GCS bucket (wasb.weights_path
default ~/models/WASB-SBDT/...). The config selects these but does not provision them; packaging
must solve weight download + ffmpeg bundling + a torch install step the config layer doesn't
cover.",
    "cloud-serving is not runnable from a downloaded app without the user's own GCP project,
a built+pushed Cloud Run worker image, a GCS bucket, and ADC -- none provided by config or app
(M7 still owner-gated/unbuilt). Marketing 'your machine or our cloud, switch anytime' (D13)
overstates what a downloaded build delivers for the cloud half today.",
    "truly-local's preset sets skip_ai_handoff=True (zero-cloud), so by default it emits the
LOCAL detector windows AS rallies with no Gemini refinement -- quality is the raw local
detector. A user toggling to truly-local for privacy silently gets lower-quality reels unless
the UX explains the trade-off (the owned model isn't at the D11 floor yet).",
    "The legacy AppConfig.get()/__getitem__ shim + extra=allow tolerate typo'd keys (top-
level extras kept-but-unread, section typos silently dropped, only a log WARNING). A hand-edited
config.json from a non-tech user can silently no-op a setting; drive config through validated UI
writes, not free-text editing, or surface the _unknown_key_paths warnings prominently.",
    "Windows reality: truly-local detection routes through WSL
(TrackNetConfig/WasbIndexConfig default wsl_distro=Ubuntu, conda envs, ~/clips). A downloadable
Windows app cannot assume WSL2 + conda + a CUDA GPU; the native runner is Linux-only. The config
models encode a developer's WSL setup as defaults -- wrong for an end-user install, needing
profile-time overriding or a different local-compute story.",
    "Env-over-config precedence (DEPLOYMENT_PROFILE, RALLY_DETECTOR_IMPL, WASB_*) means a
stray env var on the user's machine can silently override the app's chosen profile/detector -- a
support footgun unless the launcher clears/controls the relevant env."
  ],
  "open_questions": [
    "For a downloadable app, does 'cloud-serving' mean (a) connect to a hosted Khelsutra
service, or (b) the user brings their own GCP project? The config/compute seam only supports (b)
today (CLOUD_RUN_* env + ADC); (a) needs a different StorageBackend/ComputeBackend (signed-
URL/API client) that doesn't exist yet.",
    "How are WASB weights + the WASB conda/torch env delivered in a packaged truly-local
build? wasb.weights_path defaults to a ~/models path and weights live in a GCS bucket (WASB-C3);
there is no in-app downloader/provisioner wired to config today.",
    "Should the packaged app expose a 4th 'just works' profile pinning detector_impl for a
no-WSL / CPU-or-bundled-GPU consumer machine? The current truly-local preset assumes a developer
GPU box (WSL/native/auto), not a typical end-user environment.",
    "Where does the Gemini API key entry live in the packaged UX, and is the cloud handoff
allowed in truly-local? Today truly-local forces skip_ai_handoff=True (no key) but a user may
want local-compute + cloud-Gemini-refine -- a knob combination no preset expresses.",
    "Is there a first-run wizard the owner intends to build that calls save_default_config +
suggest_profile + enforce_startup_checks, or does the SPA (khelsutra/web) own all onboarding?
The engine has the primitives but no assembled flow.",
    "Should config.json validation surface unknown-key warnings to the end user (not just
logs)? For a non-tech user a silently-dropped section key is invisible today."
  ]
},
{
  "area": "Storage + Compute backend registries (the local-OR-cloud axis)",

```

"summary": "Storage and compute are two URI/config-driven seams that let the SAME pipeline read/write either local files or cloud sources, default-OFF (today's behaviour byte-for-byte). STORAGE: a `StorageBackend` ABC (backend/storage/base.py) with four lazy-imported implementations (local/s3/gcs/gdrive) selected by the URI scheme via `ref.parse\_uri` ? a bare path or `local://` -> local (identity passthrough, no copy), `s3://` -> S3Storage (also Cloudflare R2 / DO Spaces / MinIO via endpoint_url), `gs://|gcs://` -> GCSStorage, `gdrive://` -> a path under a locally-MOUNTED Google Drive (plain local FS, no API). The API/CLI/worker boundary calls thin helpers `acquire\_local\_input` / `resolve\_input\_ref` / `fetch` / `put` / `list\_uris`; input resolution honours `storage.input\_backend` + `storage.input\_root` so a bare name can be joined under a bucket root, while an explicit scheme or absolute path always wins. COMPUTE: a `ComputeBackend` ABC (backend/compute/base.py) with a factory `get\_compute\_backend(config)` that returns a `CloudRunCompute` ONLY when `deployment.compute\_target == "cloud\_run"` and `None` otherwise (= run the worker in-process, today's path). CloudRunCompute dispatches a Cloud Run GPU **Job** via an injected `CloudRunJobClient`, then bucket-polls a tiny done-marker object (NOT the Cloud Run API status) until the worker writes it, then reads `report.json`. Credentials are NEVER stored in config ? only endpoints/regions/paths/source-selectors; boto3 / google-auth / ADC supply secrets from their own chains and env. The base install stays light because every cloud SDK (boto3, google-cloud-storage, google-cloud-run) is lazy-imported, and only the two storage SDKs are declared as optional extras.",

"implications": [

"Base download stays light by design: a packaged app can ship WITHOUT boto3 / google-cloud-storage / google-cloud-run ? `import backend.storage` and `import backend.compute` are pure-stdlib until a cloud URI/target is actually used. The local + gdrive paths need zero cloud SDKs, so a 'truly-local' user has no cloud install at all.",

"To let a packaged app point at gs://|s3://|gdrive:// from a config/UI, expose StorageConfig fields (backend, input_backend, input_root, output_root, per-backend dicts) in the UI ? these are endpoints/regions/paths only and safe to persist; the app then resolves any source via storage.acquire_local_input with no code change.",

"Credentials must be supplied OUT OF BAND, never in the app's config file: S3 via AWS_* env or ~/.aws, GCS via GOOGLE_APPLICATION_CREDENTIALS service-account JSON / ADC. A packaged 'batteries-included' app needs a credentials UX (file picker for the SA JSON setting GOOGLE_APPLICATION_CREDENTIALS, or AWS key fields written to the boto3 env/chain) that is explicitly separate from the license-clean config layer.",

"Cloud SDKs should be packaged as OPTIONAL installable components (mirroring the storage-s3 / storage-gcs extras) so the installer/UI can fetch them on demand when the user selects a cloud backend; the import failure today is a hard ImportError, so the app must catch it and prompt to install the right extra.",

"google-cloud-run is NOT declared as an extra anywhere ? if the packaged app is ever to DISPATCH to Cloud Run (cloud-serving compute), packaging must add a `compute-cloud`/serving extra (google-cloud-run) and surface the CLOUD_RUN_JOB / CLOUD_RUN_REGION / GOOGLE_CLOUD_PROJECT env as config; otherwise the cloud-serving profile fails at dispatch with a RuntimeError.",

"gdrive:// is the lightest cloud-ish source for a non-tech-savvy user: no SDK, no OAuth, no service account ? just Google Drive for Desktop mounted. The packaged app can auto-detect the mount and surface gdrive:// inputs with zero credential UX, which fits the user-friendly-UX goal; it just needs to render the existing actionable not-mounted / not-synced remedy messages.",

"Every cloud backend lands outputs at a deterministic ``<out>/outputs/<video_id>/`` layout keyed by the MD5 video_id, so a packaged app can present results uniformly regardless of where compute ran (local in-process vs Cloud Run Job).",

"License posture for new packaging deps is satisfiable: boto3 (Apache-2.0), google-cloud-storage / google-cloud-run (Apache-2.0) are all in the allowed MIT/Apache/BSD set; the credentials UX adds no weight-licensing concern (weights live in GCS, fetched via the same storage layer)."

],

"risks": [

"Cloud creds are env/ADC-chain dependent, which is fragile for a downloadable app: a user on Windows with no AWS_* env or no GOOGLE_APPLICATION_CREDENTIALS gets a deep boto3/google-auth error far from the UI; the app needs to pre-validate creds (startup checks exist per-profile but the cred chain itself isn't surfaced) and translate failures into friendly UI errors.",

"google-cloud-run is undeclared in pyproject extras and only installed out-of-band ? a packaged cloud-serving build will silently lack it until run_job raises RuntimeError. Easy to miss in a 'batteries-included' bundle.",

"CloudRunCompute writes a done-marker only on SUCCESS; a failed/hung container makes the

dispatcher's bounded poll exhaust to PolicyTimeout (rc=4) rather than report the real failure cause ? a packaged app surfacing 'timeout' would mislead users whose job actually errored.",

"S3Storage.exists() swallows ALL exceptions and returns False (treats auth/network errors as 'not found'), so a misconfigured credential could masquerade as a missing object ? confusing in a UI that distinguishes 'no such file' from 'cannot reach bucket'.",

"GDrive auto-detect only covers macOS CloudStorage paths and Windows drive letters; Linux and non-default mounts require GDRIVE_MOUNT_ROOT, so a Linux packaged user gets a hard RuntimeError unless the UI exposes the mount-root setting.",

"The signed_url capability is uneven (local cannot sign, gdrive only emulates, NotImplementedError is the contract) ? any packaged UI feature that hands a browser a signed URL must fall back to fetch_to_local, which means downloading bytes locally first; not all backends support a zero-copy browser handoff."

],

"open_questions": [

"Should a packaged app add a dedicated `compute-cloud` (google-cloud-run) extra and a credentials-management UI, or is cloud-serving out of scope for the first downloadable release (truly-local + gdrive only)?",

"How should the app persist/scope cloud credentials given the guardrail that StorageConfig must hold NO secrets ? a separate keychain/secret store, or just point users at the standard env/ADC files via a setup wizard?",

"The WASB model weights (wasb_badminton_best.pth.tar) live in a GCS bucket ? is the plan to fetch them through this same storage layer (gs:// via GCSStorage) at first run, and if so does that force google-cloud-storage into the BASE install rather than an optional extra for the truly-local default?",

"Does the packaged truly-local profile need ANY storage extra, or is the intent that local + gdrive (zero SDK) is the only out-of-the-box path and s3/gcs are user-opt-in installs?",

"Should the worker emit a FAILURE done-marker (currently a noted follow-up) before a packaged cloud-serving release, so users get the real error instead of an rc=4 timeout?"

]

},

{

"area": "Heavy runtime deps - dual-torch, WASB weights, ffmpeg, device",

"summary": "Two disjoint Python runtimes; the dual-torch split is the crux. See key_facts for the file-grounded details.",

"implications": [

"Two runtimes (main torch-optional plus an isolated WASB py3.8 torch1.11 subprocess via WASB_PYTHON); make person_detector torch lazy or guard yolo_hybrid in the segmenters init for a no-torch Gemini tier; bundle static ffmpeg (LGPL); fetch WASB weights at first run since WASB-C3 is unclear; cloud and local are already wired via profiles and storage/compute registries."

],

"risks": [

"Double-torch bundle is large and 1.11 cu113 is old; WASB weights are not redistributable (WASB-C3) which breaks the licensing guardrail; bare-name ffmpeg fails if not on PATH; eager torch import crashes a torch-less light build; conda build needs build-time network plus ToS."

],

"open_questions": [

"WASB weight size and license; is CPU-only WASB acceptable; can WASB move to torch 2.x to drop the split; which ffmpeg license build to use; does the appliance doc pick micromamba vs a pip venv."

]

},

{

"area": "Bundling the web UI into the downloadable `batteries-included` Python app",

"summary": "The operator-facing UI is a Vite + React 19 + TS + Tailwind v4 SPA that lives in a DIFFERENT repo (khelesutra/web/), not in the engine. It is built with `vite build` into khelesutra/web/dist/ (a standard static bundle: index.html + one hashed JS chunk + one CSS + self-hosted Noto woff2 fonts, ~669 KB total); dist/ is gitignored so the built bundle is NOT checked in. The SPA talks to the engine purely via SAME-ORIGIN relative `/api/\*` calls ? the only API-base config is a hardcoded `const BASE = `"/api` in web/src/api/client.ts:7, with NO absolute URL, NO consumed env var (the `VITE_ENGINE_ORIGIN` mentioned in vite.config.ts is a dev-proxy comment that the client code never reads). In dev, Vite proxies `/api` ? localhost:8000; in prod the live design (khelesutra/docs/LOCAL\_APPLIANCE\_ARCHITECTURE.md, decisions D1/D5) puts a reverse proxy in front that serves the SPA bundle and proxies `/api` to the engine on 127.0.0.1:8000 ? explicitly NOT FastAPI serving the SPA, and explicitly NOT engine

static. Meanwhile the engine TODAY serves a completely separate, legacy hand-written vanilla-JS UI (backend/api/static/{index.html,app.js,style.css}) mounted at `/static` and returned at `/` (backend/main.py:64-74) ? this is NOT dead code (it is live-mounted and the appliance doc treats its app.js as the current bundled UI), but it is the OLD UI, superseded by the khelsutra SPA, and it only hits `/api/process` + `/api/compile`. The three locale catalogs (en/kn/hi common.json, 88 lines each) are statically `import`'ed into the i18next singleton (web/src/i18n/index.ts) and bundled into the JS chunk at build time ? they ship inside the bundle, not as separate runtime files, with always-fallback to `en`. Fonts are self-hosted via `@fontsource-variable` (bundled woff2), so kn/hi render offline with no Google Fonts runtime dependency. To ship the built SPA inside the Python app as a unified origin, FastAPI can serve khelsutra/web/dist/ as static files ? this needs ZERO SPA code change because BASE is already same-origin `/api`.",

"implications": [

"Shipping the BUILT SPA inside the Python app as a unified origin (FastAPI serves khelsutra/web/dist/, proxies nothing) needs ZERO SPA code change: BASE is already same-origin `/api`. This collapses the appliance's reverse-proxy assumption into a single FastAPI process ? simpler than the doc's proxy topology, and the recommended packaging path for a downloadable app.",

"There is a CROSS-REPO build coupling: the SPA source is in khelsutra/web (separate repo, separate npm toolchain) but the Python wheel is built here. The packaging pipeline must (a) `npm ci && npm run build` in khelsutra/web, then (b) copy/vendor web/dist/ into the engine wheel as package data. dist/ is gitignored, so it cannot simply be committed and picked up ? it must be produced at package time or vendored deliberately.",

"To bundle dist/ in the wheel, add an explicit hatchling force-include (e.g. map khelsutra/web/dist -> backend/ui/ or a top-level package) in pyproject.toml [tool.hatch.build.targets.wheel.force-include]; the current config has none, so add it rather than rely on auto-detection.",

"The engine's existing static-serving code is BROKEN for an installed app: backend/main.py:65-70 use CWD-relative paths ('backend/api/static/...'). For a pip-installed app launched from any directory these won't resolve. Replace with package-relative resolution (importlib.resources / Path(__file__).parent) pointing at the vendored dist/, and add an SPA history-fallback route (serve index.html for unknown non-/api paths) so client-side ?video= routing survives refresh.",

"Decide the fate of the legacy backend/api/static vanilla-JS UI: it is the OLD console and is superseded by the khelsutra SPA. Bundling BOTH is confusing; the packaging plan should pick one (vendor the React dist and retire/replace the legacy static UI at / and /static).",

"i18n is a non-issue for packaging: en/kn/hi catalogs and Indic fonts are compiled INTO the bundle (no runtime catalog fetch, no Google Fonts call), so the localized UI works fully offline once dist/ is vendored ? aligns with the offline/local-appliance goal.",

"Single-origin packaging makes the engine's CORS '*' irrelevant from the browser (same-origin), but it does NOT add auth ? the appliance's tenancy boundary (127.0.0.1 bind + firewall + edge gate, D1/D2) still applies; a downloadable single-process app must keep the bind-to-localhost default."

],

"risks": [

"Cross-repo build dependency is fragile: the engine wheel cannot be built without a Node toolchain + the khelsutra repo checked out at a compatible commit. Version skew between the two repos can ship a SPA that calls /api endpoints the bundled engine lacks (the appliance doc lists several net-new endpoints: /api/capabilities, /api/videos, /api/reels ? not yet in the engine). Need a pinned/vendored bundle + a contract test.",

"The committed backend/api/static UI being live at / means a naive 'serve dist/ at /' change could collide with the existing serve_index route and StaticFiles('/static') mount ? must remove/replace the legacy routes, not stack them.",

"The hardcoded same-origin BASE='/api' is a strength for unified-origin packaging but a constraint: any deployment that does NOT put the engine and SPA on one origin (e.g. SPA on Cloudflare Pages + engine elsewhere) would require an SPA code change to introduce a runtime origin knob ? currently impossible without a rebuild.",

"License hygiene of the new bundling toolchain/deps must be checked: the SPA pulls @fontsource Noto fonts (OFL) and React/i18next/Tailwind ? all need to be confirmed MIT/Apache/BSD/OFL-clean per the non-negotiable guardrail before they ride inside a distributed wheel (the engine already vendors OFL Noto under backend/assets/fonts with LICENSE files, a good precedent).",

"dist/ being gitignored means there is no reproducible, auditable record of what UI shipped in a given wheel unless the build is pinned (lockfile present: package-lock.json) and the bundle hash is recorded at package time."

```

],
"open_questions": [
  "Where should the vendored SPA live inside the wheel ? a new backend/ui/ package dir, a
  top-level static_ui package, or replace backend/api/static in place? (affects the force-include
  mapping and the FastAPI mount path).",
  "Should the engine serve the SPA directly (single FastAPI process, simplest for a
  downloadable app) OR keep the reverse-proxy topology from LOCAL_APPLIANCE_ARCHITECTURE.md D1?
  The downloadable-app goal argues for FastAPI-serves-SPA; the doc currently assumes a proxy.",
  "How is the cross-repo build orchestrated at package time ? git submodule of
  khelsutra/web, a CI step that checks out khelsutra and runs npm build, or a pre-built dist
  vendored into this repo? Pinning strategy for SPA<->engine API contract.",
  "Should the legacy backend/api/static vanilla-JS UI be deleted as part of this work, or
  kept as a fallback? It is currently live-mounted and referenced by the appliance doc.",
  "Do the SPA's expected-but-missing endpoints (/api/capabilities, /api/videos,
  /api/reels, and the bucket-ingest SPA call) need to land in the engine before the bundled UI is
  fully functional, or does the current SPA degrade gracefully (it ships demo data + a ?video=
  path) for an MVP bundle?"
]
},
{
  "area": "Rally annotation + BYO training (annotate -> CSV -> train -> ship-gate), for
  packaging a batteries-included downloadable app",
  "summary": "Annotation today is NOT in-app and NOT in-browser: it is a separate, external,
  MIT-licensed VLC 3.0.x Lua plugin (`rally-annotator`, a sibling repo) that the user installs
  into their own VLC and clicks Mark START / Mark END / reason / shots to append one row per rally
  to `<video-stem>.rallies.csv` next to the video. \"Rally annotation if needed\" in-app means,
  today, \"go use the external VLC plugin\" ? the engine and the khelsutra SPA have NO annotation
  UI, and the khelsutra design doc explicitly forbids re-authoring VLC instructions in the SPA and
  forbids a \"Train now\" button (D3/D4). The annotator CSV is directly ingestible by the engine
  with NO transform: header `rally_number,start_time,end_time,ending_reason,sport,shots_count`,
  decimal seconds, and the engine reads by column name via `gt_loader.load_gt_intervals` (accepts
  start/end OR start_time/end_time) and `stub_runner.synth_trajectory_from_labels` (needs the
  *_time names). Training is an OFFLINE CLI, `python -m backend.worker train`, that pulls labels/
  (+ videos/) from a storage URI, builds per-video trajectory CSVs, assembles a manifest, shells
  out to `python -m training.gen0.harness`, runs honest leave-one-video-out + a ship-gate, and
  pushes a versioned artifact bundle (weights.json + reviewable manifest.json). It has two
  trajectory sources: `synth` (default, GPU-free CPU mock that ENCODES the labels ? plumbing-
  meaningful only, trains a tiny LogisticRegression) and `detector` (REAL shuttle trajectories via
  the native WASB runner ? GPU-gated and needing the separate conda/torch-1.11 env, in-flight).
  The licensing ship-gate is `training/gen0/harness.py::gate_verdict`, which hard-codes
  `ship=False` for ANY commercial ship today (WASB checkpoint provenance \"C3\" unresolved +
  F1@tIoU target not calibrated, issue #172), and the bundle manifest attests
  `no_paid_llm_training_labels: True` and `label_source: \"human (rally-annotator golden CSVs)\"`.
  There is currently NO serving path that consumes a trained Gen-0 bundle ? the per-user serving
  bridge (`personalization/serving.py`) is explicitly \"NOT WIRED INTO THE SERVED PATH\".",
  "implications": [
    "Annotation cannot be bundled into the Python app as-is: it is a VLC Lua extension in a
    SEPARATE MIT repo requiring VLC 3.0.x (4.0 broke the Lua API). A batteries-included app must
    either (a) ship/instruct the VLC plugin install as an external step (the current khelsutra D3
    stance: link out, don't re-author), or (b) build a NEW in-browser/in-app annotator that emits
    the SAME `rally_number,start_time,end_time,ending_reason,sport,shots_count` CSV ? the roadmap
    explicitly lists an 'HTML5 <video> page that exports the same CSV' as future work (rally-
    annotator/README.md:112-113). Bundling the existing plugin = also bundling VLC, a heavyweight
    system dep.",
    "The packaged app CAN ship the full annotate->train->bundle CLI for the GPU-FREE synth
    path with zero extra heavy deps: it needs only numpy/scipy (BSD) + ffmpeg/ffprobe (system dep,
    already required) + the existing backend/training code. This validates the whole pipeline on a
    CPU laptop/CI, but the trained model is plumbing-only (synthetic trajectory), NOT a usable rally
    model.",
    "The REAL training path is the hard packaging problem: it needs (1) a GPU, (2) the
    separate conda env (torch 1.11+cu113) for native WASB driven by
    WASB_PYTHON/WASB_WEIGHTS/WASB_REPO_DIR, and (3) the WASB weights `wasb_badminton_best.pth.tar`
    (license-clean MIT WASB, but the *.pth.tar checkpoint provenance is the unresolved C3 blocker).
    A downloadable app must orchestrate this dual-env / weight-download (the decoupled-compute
    storage seam already supports gcs:// fetch) or punt real training to a cloud profile.",

```


"The licensing guardrail is enforced at the ARTIFACT level, not at runtime: gate_verdict + the manifest attestation. A packaged app that offers 'train your own model' must surface gate_verdict honestly (ship=False today) and must NOT let a user ship/serve a commercial model ? there is no UI for that yet, and the gate refuses it. The app should treat the bundle's manifest.json as the reviewable contract and never auto-promote a model into serving.",

"Nothing consumes a trained Gen-0 bundle at serving time yet (personalization/serving.py is groundwork, not wired). So 'train -> use my model' is NOT an end-to-end CUJ today ? a packaged app can offer annotate + train-to-bundle, but 'serve the trained model' is an unbuilt, owner-gated step. Don't advertise a closed loop the engine can't run.",

"Model weights are not shipped in the repo (only artifacts/.../manifest.json is committed; weights.json is generated and *.pt is gitignored; WASB .pth.tar lives in GCS). A packaged app must FETCH weights (WASB checkpoint from the bucket via the storage backend) at first run rather than embed them ? and must keep every fetched weight on the MIT/Apache/BSD allowlist, with the C3 provenance caveat unresolved.",

"The video<->label join is MD5-based: a packaged app that re-encodes / transcodes user uploads before annotation or training will break the label join. The app must preserve byte-identity between the annotated file and the trained file, or re-key carefully."

],

"risks": [

"Bundling the existing annotator means a hard dependency on VLC 3.0.x specifically (VLC 4.0 changed the Lua API and is unsupported), which is fragile and a heavy external system dep ? and it is desktop-only, not in-browser, conflicting with the 'self-hosted web service' product direction (D10).",

"The 'real' training path is effectively unrunnable inside a simple downloadable app: it needs a GPU + the second conda/torch-1.11 env + WASB weights. Offering 'train your own model' risks misleading non-technical users into expecting one-click real training when only the synth (mock) path is turnkey.",

"The commercial ship-gate is hard-False today (C3 unresolved + uncalibrated F1 target #172). Any packaged 'BYO model' feature that implies a shippable/servable model would contradict the gate and the licensing guardrail. Serving a trained bundle is unbuilt and owner-gated.",

"The synth-path 'trained model' is meaningful only as plumbing ? a user could mistake the LOVO F1 numbers it prints for a real accuracy claim. The khelsutra doc already forbids accuracy-superiority claims (D5); the app's UX must mirror that honesty.",

"ffmpeg/ffprobe are system deps the train path silently relies on (fps/frame_width probe). On a box without ffprobe the detector training SKIPS videos (won't guess) ? a packaged app must guarantee ffmpeg is present or training silently degrades.",

"Two CSV schemas exist (the annotator's light schema vs the Roora vendor schema); conflating them in packaging/UI could corrupt the label pipeline."

],

"open_questions": [

"For a downloadable app, does annotation become a NEW in-browser annotator emitting the same CSV (the README's future 'HTML5 <video>' idea), or does the app keep delegating to the external VLC plugin (khelsutra D3)? This is unresolved and owner-gated.",

"Will the packaged app ever expose the real (detector/GPU) training path, given it needs the dual conda env + GPU + WASB weights, or will real training only run against a cloud/GCP serving profile while local stays synth-only?",

"Is there any intent to wire a trained Gen-0 bundle into serving (personalization/serving.py -> fusion_hybrid._select_for_handoff)? Today the loop is open; 'use my model' is not an end-to-end CUJ.",

"How will the app fetch/cache the WASB checkpoint (GCS) within the licensing allowlist while the C3 checkpoint-provenance blocker is unresolved? Can a license-clean checkpoint be shipped at all?",

"Does the engine need a NEW PATCH /api/segments correction endpoint (khelsutra P8, designed/owner-gated) to grow the corpus in-app, given there is no annotate/correction API today?",

"Should the packaged app surface gate_verdict (ship=False, blockers) to the user so 'train your own model' is honest about not being commercially servable yet?"

]

},

{

"area": "prior appliance and desktop design",

"summary": "Owner D10 superseded the desktop-app plan with a self-hosted webserver appliance the user spins up; compute and storage are settings; the engine stays auth-less behind an external edge gate that is the only tenancy boundary.",

"implications": [

```

    "Ship engine plus SPA plus a bundled edge-auth gate as one artifact",
    "Lift the Cloud Run dual-env Dockerfile; a container is easiest",
    "WASB weights need runtime fetch plus a license gate",
    "Compute-as-settings needs a net-new capabilities endpoint"
  ],
  "risks": [
    "No-Dockerfile claim is stale and DOC_STATUS misses the supersession",
    "Desktop P0 on a no-GPU laptop was never measured",
    "WASB weight provenance blocks public redistribution",
    "Auth is structural-only so enforce localhost plus firewall plus gate"
  ],
  "open_questions": [
    "Edge-auth Caddy basicauth vs cloudflared plus Cloudflare Access",
    "GPU-host-only or no-GPU laptop reviving P0",
    "How to deliver WASB weights publicly",
    "Container vs native vs PyInstaller; net-new videos and reels endpoints"
  ]
},
{
  "area": "Completed end-to-end CUJs the packaged \"batteries-included\" app MUST preserve",
  "summary": "Across the engine (FastAPI + worker CLI + single-shot CLI) and the khelsutra
web/ SPA, a set of end-to-end Critical User Journeys is already DONE and verified, and the
downloadable app must keep every one of them working. The flagship loop is fully shipped: point
at a video (local upload OR a gs:// bucket URI entered IN the SPA) -> engine
fetches/validates/segments via Gemini -> SPA loads rallies via /api/query (default all-selected)
-> deterministic query bar + per-rally checkbox selection sharing one Set -> /api/compile
stitches selected segments -> reel previews in <video> and downloads to the local machine; this
ran live both en and hi (and kn works via the same switcher). The SPA is fully trilingual
(en+kn+hi) with automatic browser-language negotiation, self-hosted OFL Noto fonts, offline/air-
gap renderable, proven by a Playwright screencast. On the engine side, the worker CLI does GCS-
in -> WASB GPU inference -> GCS-out (22 rallies on a real L4), and the cold-start GPU
provisioning (main torch 2.6 + separate wasb conda torch-1.11 env + staged weights) is validated
end-to-end. The decoupled-compute seam (profiles/storage/compute registries, startup checks,
cost ledger, Cf-Access edge-auth) is merged but DEFAULT-OFF. The annotate->train loop is only
PARTIALLY real: the MIT rally-annotator (VLC) produces ingestible CSVs and worker train runs,
but the GPU-free training path is a pipeline-validation MOCK and the real-model ship-gate
currently refuses a commercial ship. The packaging effort must preserve these journeys without
requiring git clone, while solving ffmpeg/system-deps, the dual torch envs for native WASB, GCS-
hosted weights, and bundling the cross-repo SPA.",
  "implications": [
    "The reel-build loop (process -> jobs -> query -> compile -> highlights) is the spine of
CUJ-1/2/3/4 and runs on the in-process FastAPI worker with storage=local; the package must ship
this loop runnable with ZERO config on a CPU box (Gemini or motion), so ffmpeg/ffprobe must be
bundled or auto-detected and GEMINI_API_KEY supplied by the user at runtime.",
    "The packaged endpoint must invoke the server such that backend.main module state
(db_instance) is initialized for the job worker ? do NOT reintroduce the __main__-vs-import
split that caused #281 (jobs hung in queued). The console-script `indexer` and `python -m
backend.main --server` must both yield a working async drain.",
    "Native WASB (CUJ-6/7, best quality) requires a SEPARATE Python env (conda, torch
1.11+cu113) alongside the main torch 2.6+cu124, plus GCS-hosted weights
(wasb_badminton_best.pth.tar) and a GPU. A 'batteries-included' download cannot bundle both CUDA
stacks naively; package the GPU/WASB path as an OPTIONAL, separately-provisioned component (the
deploy/cloudrun Dockerfile + gpu_setup.sh recipe is the reference), and ship a working CPU
default (Gemini/motion) so the app is useful out-of-the-box without a GPU.",
    "A no-Gemini-key fully-local run MUST set indexing.skip_ai_handoff=true (the truly-local
preset does this) or WASB returns 0 rallies ? the packaged truly-local profile must default this
on.",
    "The trilingual SPA (CUJ-5) is air-gap renderable with self-hosted OFL Noto fonts and
lives in khelsutra/web (a different repo). The package must bundle a BUILT SPA (npm run build
output) served as the UI, NOT the dead engine backend/api/static; this is a cross-repo build
step the packaging pipeline must own, and the bundle must keep the offline/no-CDN guarantee.",
    "torch/torchvision are deliberately absent from pyproject deps (CUDA/CPU wheels need
--index-url, which PEP621 can't express). The installer/launcher must resolve torch out-of-band
(a post-install step or a wheel pin per platform) for any real-detection path; the package can't
rely on `pip install` alone pulling a usable torch.",

```

"GCS/S3/Drive ingest (CUJ-2 cloud variant) needs google-cloud-storage (storage-gcs extra) + ADC credentials; the package must make these OPTIONAL extras and degrade gracefully to local-upload when absent, mirroring the current StorageBackend registry + extras layout.",

"The annotate->train USP (CUJ-9) is only partially real: the package should expose the worker train CLI and document the MIT rally-annotator + CSV bridge, but must NOT present one-click real-model training (synth=mock; real=GPU+in-flight native WASB+owner-gated ship-gate). Honesty fences must survive into the packaged UX.",

"All cloud/auth/cost serving features (edge-auth, cost ledger, cloud_run compute, profiles) are DEFAULT-OFF; the downloadable app's default profile should remain truly-local single-user so the package is byte-identical to the verified local CUJs unless the user opts into cloud/hosted mode.",

"Edge-auth is the ONLY tenancy boundary for any non-localhost exposure (engine has zero app auth + CORS *). If the package can be exposed beyond 127.0.0.1, it must ship/document the gate (Cf-Access or reverse-proxy basicauth) + rotate keys + set security.allowed_video_root; default-bound to localhost otherwise."

],

"risks": [

"The packaged single-entripoint must not reintroduce the backend.main __main__-vs-import module split (#281) ? a packaging launcher that runs the app differently than the tested `python -m backend.main --server` could silently hang every job in queued.",

"Bundling the cross-repo khelsutra/web SPA build into the engine package risks drift: the SPA expects /api contracts (play_segments, download_url, /api/jobs result.video_id) ? a stale bundled UI would break CUJ-1/2/3 just like the dead backend/api/static app.js already does.",

"Native WASB cloud GPU runs (CUJ-6/7) were verified on Singapore L4 with skip_ai_handoff=true; the exact TUNED config that reproduces 22 rallies on a fresh build was previously flagged as a missing piece ? packaging a 'native' profile risks shipping a config that yields 0 rallies if skip_ai_handoff/detector_impl wiring is wrong.",

"kn/hi catalogs are AI-DRAFTED with native-speaker review still PENDING ? shipping the trilingual UI to non-test users before review is a quality/trust risk the owner flagged.",

"Two torch CUDA stacks + GCS-hosted weights make a true 'download and run with GPU' artifact large and fragile; an over-ambitious all-in-one bundle could be unshippable, while a CPU-only bundle silently loses the best-quality (WASB) CUJ if not clearly surfaced.",

"The annotate->train CUJ is easy to overstate; packaging copy/UX could imply a working owned model when the ship-gate refuses commercial ship and synth training is a mock ? violating the standing honesty + licensing guardrails.",

"GCS/Gemini paths incur paid + cloud-upload costs with no per-video hard cost ceiling in the default path (billing ledger exists but is default-OFF and currently a no-op until usage emission is wired) ? a packaged app that defaults users into Gemini could surprise-bill them."

],

"open_questions": [

"Does the packaged app ship the BUILT khelsutra/web SPA bundle as the served UI, and which repo's CI owns producing/versioning that bundle so it stays in sync with the engine /api contract?",

"Is native WASB (GPU) an in-scope CUJ for the downloadable app's first release, or is the batteries-included default explicitly CPU-only (Gemini/motion) with GPU/WASB as a documented optional add-on?",

"What is the exact, reproducible truly-local native-WASB config/command (the one that produced 22 rallies) that the packaged 'native' profile should bake in, given the prior cold-start config-drift that yielded 0 rallies?",

"How does the package resolve torch/torchvision (post-install hook? platform-pinned wheels? prompt the user?) given they are deliberately excluded from pyproject deps?",

"Should the packaged worker train CLI + rally-annotator linkage be included at v1, or deferred, given the train path is a mock/GPU-gated and the ship-gate refuses commercial ship?",

"Does the package need to bundle ffmpeg/ffprobe binaries per-platform, or assume them as a documented system prerequisite (current behavior)?",

"For any cloud (GCS/Cf-Access/cost) feature in the package, do they remain strictly default-OFF opt-ins, and is exposure beyond localhost in-scope (requiring the bundled edge gate) or explicitly out-of-scope for v1?"

]

},

{

"area": "Verification + licensing constraints on packaging",

"summary": "Testing today is a fully-offline, fully-mocked pytest suite (110 test_*.py files) launched via run_all_tests.py, which is just a thin pytest wrapper that defaults to -q

and reports PASS/FAIL by pytest exit code; there is NO real-video/GPU/network smoke test in CI (those are explicitly handed to the owner per CLAUDE.md and LOCAL_APPLIANCE L5). CI (.github/workflows/ci.yml) runs five lanes on Python 3.13: (1) ruff lint over the whole repo, (2) mypy on backend+training against a lenient baseline with a 14-entry quarantine, (3) an import-smoke lane that deliberately does NOT install cv2/torch/numPy and stubs them, (4) the full-suite lane that installs CPU-only torch out-of-band then pip install -e .[dev] and enforces --cov-fail-under=70 over backend+training, and (5) a cross-OS (ubuntu/windows/macos) golden-fixtures determinism guard. The coverage floor is 70% (calibrated 2026-06-11 against the obsreport-absent lane's measured 72%; local-with-obsreport ~73%) and must never go down without an owner decision. Any packaging change must keep all five lanes green: notably the import-smoke lane proves backend.main imports with the GPU stack absent, and test_import_smoke.py enforces the ADR-008 "import wall" (serving never imports training) plus a pure-numPy/scipy featurizer constraint ? packaging must preserve these module boundaries. The licensing rule is human-policy-enforced (MIT/Apache-2.0/BSD only for code, data, AND weights; paid LLMs = inference/serving only, never a training source) with NO automated license-scanner in CI; the only code-enforced gate is the de-attribution/provenance refusal gate in the golden-fixtures + training-harness path (RefusalError, no_paid_llm_training_labels=True attestation), not a dependency scanner. On packaging tools: the live design (DESKTOP_APP_PLAN.md, superseded as a product surface by D10's self-hosted-webserver but its licensing analysis still authoritative) already locks D4 ? ship an LGPL ffmpeg build using openh264 (BSD) or OS hardware encoders, never statically-linked libx264 (GPL), which is the single biggest licensing landmine in the bundle. PyInstaller is named as the chosen backend bundler; conda's Anaconda-ToS friction is already real and worked around in deploy/cloudrun/Dockerfile (explicit `conda tos accept`),

"implications": [

"The packaged app must ship an LGPL ffmpeg build (openh264/BSD or OS hardware encoders), NOT a libx264-statically-linked GPL build, or the whole bundle becomes GPL-contaminated ? this is the dominant licensing constraint on any downloadable artifact (DESKTOP_APP_PLAN D4).",

"PyInstaller is BSD/MIT-licensed (its bootloader is GPL-with-an-exception that explicitly permits proprietary/closed-source frozen apps) so it is license-compatible for a downloadable app. Nuitka is Apache-2.0 (compatible). micromamba (BSD) is the license-clean way to ship the conda-based WASB env WITHOUT the Anaconda Distribution ToS that gates conda/Miniconda. Docker Desktop has commercial-use ToS for larger orgs, but Docker Engine/the OCI images themselves are Apache-2.0 and license-clean for distribution.",

"The dual-torch problem (main env torch 2.6+cu124 vs WASB conda env torch 1.11+cu113) means a single PyInstaller freeze cannot cover both; the precedent (Cloud Run Dockerfile) keeps WASB in a separate conda env invoked as a subprocess via \$WASB_PYTHON ? the downloadable app needs the same two-env shape (e.g. micromamba-provisioned WASB env beside a frozen main app).",

"Any new packaging dependency (the freezer, an installer framework, a license-clean ffmpeg distribution, micromamba) must itself be MIT/Apache-2.0/BSD ? and since there is no automated license scanner, this must be human-verified per dependency before adding it.",

"scipy must be added as an EXPLICIT pyproject dependency before packaging ? it is serving-required (backend/features) yet currently only transitive via supervision; a frozen bundle or a supervision bump could silently omit it and break inference.",

"The obsreport git+ssh private-repo dependency cannot be resolved by a non-repo downloading user (no SSH key/access); it must stay a truly-optional extra (it already degrades to a no-op via pytest.importorskip) and must NOT be bundled into the downloadable app.",

"WASB weights (wasb_badminton_best.pth.tar) cannot be baked into the bundle: WASB-C3 provenance (academic-data-trained) is unresolved for commercial sharing ? they must be fetched at runtime from a bucket, exactly as the Cloud Run image does (WASB_WEIGHTS env, runtime stage).",

"Packaging verification can ride the existing offline-mocked suite + the import-smoke lane (which already proves GPU-stack-absent import and the training import wall); a NEW packaged-artifact smoke test (e.g. `indexer --help` / server boot on the frozen binary) would be net-new since CI today only tests source, not a built artifact. Any change must keep all 5 CI lanes green and coverage >= 70%."

],

"risks": [

"GPL contamination via libx264 in the bundled ffmpeg is the highest-severity (legal) risk; mitigated only by deliberately choosing an LGPL ffmpeg + openh264/hardware encoders ? easy to get wrong because the current dev stitch path uses libx264.",

"No automated license gate in CI means a license-incompatible packaging dependency (e.g. a GPL installer helper, an Anaconda-ToS-encumbered conda channel) could slip in unnoticed; the guardrail is purely human review.",

"Anaconda Distribution ToS: using conda/Miniconda defaults (repo.anaconda.com main/r channels) to build the WASB env carries commercial-use ToS exposure for orgs >200 employees; the

Dockerfile sidesteps it only by auto-accepting the ToS, which is a posture, not a license clearance ? micromamba + conda-forge is the cleaner path.",

"scipy being undeclared (transitive-only) is a latent break: a future supervision pin/version that drops scipy, or a freeze that prunes unreferenced transitive deps, silently removes a serving dependency.",

"Two incompatible torch stacks make a single-binary freeze infeasible; underestimating this (treating it as one PyInstaller target) would produce a broken bundle ? the separate-env subprocess shape is mandatory.",

"A frozen/bundled artifact is NOT exercised by today's CI (which tests source on Python 3.13 + CPU torch); behavior of the frozen binary (hidden imports, data files like backend/assets/fonts, namespace-package resolution since backend has no __init__.py) is unverified until a real built-artifact smoke test exists.",

"WASB-C3 weight-provenance is ship-blocking for any non-owner serving (gate_verdict hard-codes ship=False); a downloadable public app that bundles or auto-fetches those weights without C3 resolution would violate the licensing guardrail."

],

"open_questions": [

"Is the freezer choice PyInstaller (named in DESKTOP_APP_PLAN) or a frozen-venv/micromamba sidecar, given D10 pivoted from desktop-app to self-hosted-webserver ? does the webserver framing reduce the need to freeze at all (could be `pipx install` / a downloadable wheel + a bootstrap that provisions the WASB env)?",

"Where will a license-clean LGPL ffmpeg build come from per-OS (Btbn LGPL Windows builds? Homebrew/apt? a bundled redistributable), and who verifies it excludes GPL encoders?",

"Will the downloadable app ship the WASB env via micromamba+conda-forge (license-clean) to avoid the Anaconda Distribution ToS, and is that decision recorded anywhere yet?",

"Should scipy (and the implicit transitive set) be pinned/declared explicitly before any freeze, and is there appetite to add an automated license-scan CI lane (pip-licenses/reuse) given the guardrail is currently human-only?",

"Should CI gain a built-artifact smoke lane (freeze, then run `indexer --help` / boot the server) so packaging regressions are caught, or does that stay an owner-side manual check like the GPU/real-video paths?",

"Does the coverage floor (70%) need to account for net-new packaging/bootstrap code, or will that code be excluded from --cov to avoid dragging the floor?"

]

}

],

"research": [

{

"topic": "distribution-channels",

"recommendation": "Lead with (c) uv tool install, fronted by a generated one-line install script, and back it with (d) a private/direct wheel source; keep (a) PyPI-wheel+pip as the documented power-user fallback and ship (e) an offline CPU bundle for the field/air-gapped persona. Reasoning: the dominant non-tech-Windows barrier is \"does the user have a correct Python and can they type the torch --index-url line.\" uv is the only channel that (1) installs itself in one PowerShell line with automatic PATH, (2) downloads a correct Python 3.10+ so the user needs nothing pre-installed, and (3) is fast and per-app isolated. Because `--torch-backend=auto` and `tool.uv.index` do NOT work in `uv tool install`/`uvx` (they are `uv pip`/project-only), do NOT rely on `uv tool install <app>` to also fetch torch. Instead ship a thin generated installer (install-indexer.ps1) that: (i) bootstraps uv, (ii) runs `uv tool install --from <your-wheel-url-or-private-index> badminton-highlight-indexer`, (iii) installs the matching torch into that tool's environment using `uv pip install --torch-backend=auto torch torchvision` against the tool's venv (or, in a uv project layout, sets tool.uv.index+sources and runs uv sync), (iv) drops ffmpeg/ffprobe next to the venv and exposes them on PATH, (v) on first `indexer --server` run fetches wasb_badminton_best.pth.tar from GCS. The script makes the torch index-url problem invisible ? the user double-clicks one .ps1. Do NOT pursue PyInstaller onefile for the GPU path (2.6-5GB, brittle, still can't host the WASB torch-1.11 conda env). The WASB second-env stays a separate concern: gate real local-GPU detection behind a SEPARATE managed env (conda/uv venv #2) or a container, and ship the truly-local default as Gemini/stub/CPU-torch so the one-click install always yields a working appliance even on a CPU-only Windows box.",

"fit": "Dual-torch reality (engine torch 2.6+cu124 vs WASB conda torch 1.11+cu113, per LOCAL_APPLIANCE_ARCHITECTURE.md:96) is the decisive constraint: NO single-venv channel (pip/pipx/uv tool) and NO single PyInstaller bundle can host both ABIs ? so the distributable must install the MAIN engine env one way and provision the WASB env as a separate managed venv/conda/container, exactly as the appliance doc already treats native WASB as a separate GPU-host concern. The ffmpeg/ffprobe system-dep gap (none of pip/pipx/uv deliver it) must be solved

by the installer bundling a license-clean (LGPL-shared or BSD) ffmpeg build ? and that ffmpeg binary is itself a new \"dependency\" subject to the MIT/Apache/BSD-only guardrail, so vet it. uv's auto-Python-download directly serves the non-tech India coach on Windows who likely has no Python; the GCS-hosted weights (wasb_badminton_best.pth.tar) fit a first-run bootstrap, and a private/direct-wheel source (d) keeps the app unlisted to match the commercialization/hosted-product posture rather than publishing to public PyPI. The existing hatchling wheel + extras (storage-s3/storage-gcs/auth, console script indexer=backend.main:main) need ZERO repackaging to feed any of these channels ? only torch, ffmpeg, the WASB second env, and the SPA bundle (a cross-repo khelsutra/web concern) are out-of-band. The delivered artifact must preserve every shipped CUJ: `indexer --server --port 8000` (FastAPI loop), `python -m backend.worker {infer|train}`, and the single-shot CLI with --segmenter {gemini|yolo_hybrid|wasb_hybrid|motion} ? all of which are console-script/module entrypoints any of these channels expose unchanged.",

```
"refuted": []
```

```
},
```

```
{
```

```
"topic": "containerization",
```

```
"recommendation": "Adopt a TWO-VARIANT Docker strategy and make it the appliance unit ?
```

but lead with the small one. (1) DEFAULT = a CPU-only image (Option C base): python-slim/ubuntu + ffmpeg + `pip install . torch --index-url ../cpu` + FastAPI + in-proc worker + bundled SPA, ~1-2 GB. This honestly covers the shipped Gemini/stub/motion CUJs that need NO GPU, which is the realistic state of a non-tech coach's Windows laptop. (2) OPT-IN `gpu` compose profile = the heavy CUDA image, built by EXTENDING the existing deploy/cloudrun/Dockerfile (Option A): same two-Python-env pattern (main cu124 + a micromamba `wasb` cu113 env reached via \$WASB_PYTHON), GPU via Compose `reservations.devices`, weights mounted at runtime. Do NOT split the WASB detector into its own container (Option B): the engine's worker is in-process and the detector is a subprocess call, so a separate container is net-new RPC plumbing with no payoff on a single-GPU box. Make `/api/capabilities` (already net-new in the design, T7) drive the SPA so a CPU box never offers WASB. Concretely, the compose file gets ONE app service + a `gpu` profile that swaps the image and adds the devices block; SQLite goes in a NAMED VOLUME, the user's videos + weights in bind mounts/volumes kept on the WSL2 side (never a /mnt/c Windows path) for I/O. Two engine prerequisites this forces and must be filed: (a) the server must bind 0.0.0.0 inside the container (today it hard-binds 127.0.0.1 at backend/main.py:735) with the firewall/edge gate as the safety boundary; (b) the dual-torch image needs a documented out-of-band torch install step since pyproject omits torch by design. Flag the honest fork (Option D): for a truly non-technical Windows user, Docker Desktop + WSL2-GPU is itself heavy friction and conflicts with owner decision D10 ('self-hosted webserver the user spins up, NOT a desktop app') ? so position Docker as the REPRODUCIBILITY/cloud-and-power-user unit, and keep a native `indexer --server` launch path for the lightest local case. Track the Docker Desktop commercial-licensing threshold against the repo's non-negotiable licensing guardrail.",

```
"fit": "The dual-torch problem is the ONE thing Docker solves cleanly here and the repo
```

already proves it: deploy/cloudrun/Dockerfile co-locates the main cu124 env and the WASB cu113 env in a single image with the \$WASB_PYTHON subprocess seam (Dockerfile:54-57) ? so 'two envs, one image' is not theoretical, it's shipped and M7-owner-run. ffmpeg/ffprobe (system deps) are a one-line apt install in that image (Dockerfile:27-30), which is far cleaner than asking a Windows coach to install ffmpeg by hand. The GCS-hosted weights (wasb_badminton_best.pth.tar) are deliberately NOT baked (WASB-C3 licensing AMBER, Dockerfile:14-16), so the appliance MUST mount or fetch them at runtime ? a first-class volume in the compose file, and a licensing-gate the guardrail demands. FastAPI+SPA: the engine serves /api and a (likely-dead) backend/api/static today, but the live SPA is in khelsutra/web ? bundling it is a cross-repo build step that lands as static assets in the app image; the server's 127.0.0.1 hard-bind (main.py:735) must become 0.0.0.0 inside a container with the edge gate as the boundary (the design's auth invariant, LOCAL_APPLIANCE_ARCHITECTURE.md §4.4). The non-tech-India-coach-on-Windows reality is decisive: most such users have NO NVIDIA GPU, so the CPU-default + Gemini path (the engine's actual default_segmenter) is the honest first-run, and the multi-GB CUDA image + WSL2-GPU paravirtualization + nvidia-container-toolkit chain is reserved for the GPU opt-in profile ? keeping the common-case download small and the GPU friction off the critical path. Compose maps directly onto the engine's decoupled seams (storage=local default, compute=local, profiles/presets) so 'compute/storage as a setting' (D6/D8) becomes env-vars + profiles, not forks. Licensing: micromamba (over Miniconda) sidesteps the Anaconda-defaults ToS friction the gpu_setup.sh already had to auto-accept, and Docker Desktop is free at this user's scale but its >250-employee/\$10M commercial threshold is a flag to watch as KhelSutra commercializes.",

```
"refuted": []
```

```
},
```

```
{
```

"topic": "frozen-binaries",
 "recommendation": "VIABLE ? but ONLY as a CLI/server appliance (matching owner decision D10), tiered, and NOT as a single self-extracting onefile. (1) GEMINI-ONLY / no-torch tier = VIABLE NOW, low-risk: without torch the dependency set (fastapi, uvicorn, opencv-python-headless, numpy, pydantic, google-genai, supervision) freezes cleanly; bundle stays modest (tens of MB + ffmpeg). Use PyInstaller ONEDIR (not onefile ? avoids AV false positives), ship ffmpeg LGPL binaries side-by-side, wrap the existing indexer/backend.main --server entry (already freeze-friendly: single-process uvicorn on 127.0.0.1, main.py:735). This is the recommended FIRST shippable artifact. (2) FULL LOCAL-MODEL tier (WASB) = PARTIALLY viable / higher-risk, defer. The blocker is NOT the freezer ? it is the dual-torch-env design: WASB runs in a separate torch-1.11 conda/WSL interpreter the engine shells out to (wasb_runner.py/native_wasb_runner.py). No freezer bundles two interpreters in one process. Two honest paths: (a) freeze the WASB env as its OWN sidecar binary spawned as a subprocess (doable, but doubles the torch payload to ~1GB+ and doubles signing/AV surface), or (b) the strategically right move ? finish the de-WSL native runner AND export WASB to ONNX, ship ONE onnxruntime (no torch), collapsing to a single freezable env and slashing size. Path (b) ties to the existing OWNED_MODEL_IMPLEMENTATION_PLAN export-clean mandate and the DESKTOP_APP_PLAN P0 spike. Tooling pick: PyInstaller (onedir) as default ? biggest recipe base for this exact stack, clean GPL-with-exception license, fast builds. Keep Nuitka (Apache-2.0, built-in torch/onnxruntime pruning) as fallback if size/IP-hardening matters once ONNX lands. Reject cx_Freeze (documented torch DLL pain, thinner recipes) and Briefcase (explicitly weak on C-extension/ML deps). Regardless of tool: budget per-OS code-signing (Win Authenticode/EV, Mac Developer-ID + Hardened Runtime + notarytool + stapler) and a license audit (LGPL ffmpeg only) as separate non-trivial workstreams.",

"fit": "Our engine is unusually freeze-FRIENDLY in the parts that usually break freezers, and freeze-HOSTILE in exactly one place. FRIENDLY: backend/main.py:735 already runs uvicorn single-process with the app object on 127.0.0.1 (no workers/reload/import-string) ? the textbook PyInstaller pattern; cloud SDKs (boto3, google-cloud-storage) import LAZILY in backend/storage/*, so the default local profile freezes without dragging them in; the PEP-621 indexer console entry (backend.main:main) and python -m backend.worker {infer|train} give clean freeze entry points; deployment.profile/PROFILE_PRESETS (truly-local / cloud-serving / cpu-mock) already express the exact TIERS to freeze. HOSTILE: the WASB shuttle model is a SECOND torch env (torch 1.11+cu113) reached by shelling into WSL conda activate wasb (wasb_runner.py:74) or a separate python_bin (native_wasb_runner.py:117) ? a frozen binary has one embedded interpreter and no external python/conda, so the full-local tier cannot be one frozen process; it forces either a second sidecar binary or (better, already on the roadmap) an ONNX export so only onnxruntime ships, and the GCS-hosted weights (wasb_badminton_best.pth.tar) are fetched at first run, not baked in. ffmpeg/ffprobe fit the guardrail only as side-by-side LGPL binaries (D4 ?openh264/HW encoders, never GPL libx264) since no freezer bundles them. Freezer licensing is guardrail-clean: PyInstaller (GPL-2.0 + bootloader exception) and Nuitka (Apache-2.0 core; Commercial optional) both let us ship MIT-licensed binaries without inheriting their license. Matches owner decision D10 (LOCAL_APPLIANCE_ARCHITECTURE.md:39): a self-hosted webserver+UI the user spins up ? freeze a CLI/server appliance the khelsutra React SPA talks to over localhost, not a GUI desktop app. For the non-tech-savvy India coach on Windows, ship the Gemini-only onedir installer FIRST (small, signed, no GPU, no second env); gate the heavier local-model tier behind the ONNX-export work.",

"refuted": [

"Current Nuitka (v4.x, e.g. 4.1.3) is AGPL-3.0-or-later, NOT Apache-2.0 (Apache-2.0 was only Nuitka 0.4.0?3.9). However, a GCC-style runtime exception means your COMPILED OUTPUT does not inherit Nuitka's AGPL ? you can distribute closed binaries freely ? and Nuitka Commercial is an optional paid add-on, not required to ship binaries. The licensing of the Nuitka tool itself should be recorded as AGPL-3.0+ (with binary runtime exception), not Apache-2.0."

]

},

{

"topic": "dual-python-env",

"recommendation": "Layer two decisions, do NOT pick a single bullet.\n\n1) PACKAGING TIERING = (f). Ship a torch-FREE LIGHT tier as the default download: google-genai is already a hard dep, so Gemini + motion + stub deliver the full ingest -> rallies -> reel CUJ with zero torch, a tiny install, and a reliable download for the non-tech India-coach persona on a slow uplink. The on-device WASB detector is an OPT-IN HEAVY add-on. This preserves every completed CUJ and matches the appliance's already-shipped 'runs on Gemini today' reality (LOCAL_APPLIANCE_ARCHITECTURE D10).\n\n2) For the HEAVY tier, the cleanest long-term answer is (d') EXPORT WASB TO ONNX + onnxruntime ? which the codebase was explicitly designed for (base.py names the ONNXRunner; native_wasb_runner.py is the sibling that already shells out to the same

CSV contract). It DISSOLVES the dual-env problem entirely: the WASB path needs neither torch 1.11 nor torch 2.x, ships a ~tens-of-MB runtime instead of a 5 GB framework, and gives CPU + NVIDIA(CUDA) + Windows(DirectML) + Apple(CoreML) from one artifact ? directly answering the no-GPU-laptop and Apple-Silicon unknowns the DESKTOP_APP_PLAN P0 spike was created to resolve.\n\n3) BRIDGE / fallback while ONNX export is validated: collapse to one env via (d) torch-2.x port if the timm/HRNet load is quick, OR ship the second env via (b) micromamba-from-conda-forge (license-clean, sidesteps the Anaconda `defaults` ToS) reusing the EXISTING subprocess seam (c) unchanged ? pointing WASB_PYTHON at the micromamba env. Use conda-pack (a) only as a build-time convenience for an offline-friendly first-run (no network), but mind its OS-lock + no-relocate caveat. AVOID (e) Docker as the LOCAL default ? it requires Docker + NVIDIA toolkit, a non-starter for the target user (keep Docker only for the CLOUD/HEAVY worker, where it aligns with the M6 Cloud Run + GPU seam).\n\nLICENSING NON-NEGOTIABLE: whatever env path is chosen, build/solve from conda-forge ONLY (never the Anaconda `defaults` channel) ? micromamba/miniforge/conda-pack tools are all BSD; the ToS trap is the channel, not the tool. WASB-SBDT (MIT), timm (Apache), onnxruntime (MIT) are all clean.\n\nBoth ONNX (d') and torch-port (d) MUST re-pass the owner byte-parity / golden-regression gate on a real GPU before production ? numeric drift across torch versions / the ONNX op set is the one real risk, and the offline mocked suite cannot prove the numbers (hand to owner with the labelled corpus).",

"fit": "Our engine is uniquely well-positioned: the dual-env split is already isolated behind a SUBPROCESS seam (native_wasb_runner.py shells WASB_PYTHON -> wasb_infer.py -> a Frame,Visibility,X,Y CSV; windowing/stitch are model-agnostic), and the DetectorRunner ABC (base.py:46-54) explicitly pre-designed an ONNXRunner that 'loads weights torch or onnx' and 'writes the exact same CSV format so trajectory_to_action_windows works verbatim'. So (d') ONNX or (d) torch-port slot in WITHOUT touching the pipeline, the FastAPI loop, or the decoupled compute/storage seams (M1-M8). FFmpeg/ffprobe stay system deps (LGPL build per D4); the wasb_badminton_best.pth.tar weights stay in GCS and download on demand (or convert to a versioned .onnx alongside them) ? fitting the existing StorageBackend(gcs) + presets profiles. For the non-tech India coach on Windows, the LIGHT torch-free tier (Gemini, already a hard dep) is the reliable default, and the appliance's compute-as-a-setting + /api/capabilities honesty already lets the UI offer only what the box can truthfully do ? so an opt-in ONNX HEAVY tier surfaces cleanly. This also retires the WSL2 requirement (D3) that the current owner-only path depends on, and the licensing guardrail is satisfied end-to-end (WASB MIT, timm Apache, onnxruntime/micromamba/conda-pack all MIT/BSD, conda-forge-only channel).",

"refuted": []

},

{

"topic": "weights-and-ffmpeg-bundling",

"recommendation": "Split the two concerns and reuse the existing seams. WEIGHTS: download-on-first-run from a public read-only GCS bucket we control (WEIGHTS-A), with Hugging Face Hub (WEIGHTS-B) as a strong lower-ops alternative. Do not vendor weights in the wheel because the PyPI 100 MiB per-file cap, the bloat for Gemini-only users, and the coupling of model cadence to release cadence all argue against it. Mechanics: a small fetcher resolves WASB_WEIGHTS by precedence of env var, then config, then a per-user cache directory; it downloads over HTTPS where a public bucket needs no google-cloud-storage dependency or credentials; it verifies a pinned SHA-256; it caches once for offline reruns; and it shows a loud resumable progress bar rather than a silent hang. This reuses GCSStorage.fetch_to_local and the WASB_WEIGHTS env seam so the detector code is untouched. Weights are license-clean because WASB-SBDT is MIT, but verify each weight file actual host and provenance before mirroring. FFMPEG: do not redistribute ffmpeg, since that is exactly where the GPL versus LGPL problem bites. Instead route every ffmpeg and ffprobe call through one resolver helper that checks a configured path or the FFMPEG_BINARY env var, then shutil.which, then a known install dir, and otherwise fails loud with the exact winget command; and have first-run setup, extending scripts/doctor.py, detect a missing binary and offer or auto-run the winget install of Gyan.FFMpeg. The full GPL build ships ffprobe, libx264, and libfreetype, so the engine stitch, watermark, and duration probe all work unchanged, and because the user installs ffmpeg we never redistribute GPL bits and stay MIT-clean. Reject imageio-ffmpeg as the primary path because it lacks ffprobe, which the engine requires, and its binary is effectively a GPL build, so it neither fully works nor is redistribution-clean. Keep the LGPL-bundle option FFMPEG-C, a Btbn LGPL build with the encoder swapped from libx264 to libopenh264 or NVENC, as a documented fallback only if a future truly zero-touch installer must avoid the winget step, noting it forces an encoder change and LGPL re-link or source-offer compliance. Net first-run for a non-tech Windows user: the installer or pip drops the app; doctor-style setup checks ffmpeg and ffprobe and, if absent, runs winget with consent and progress; and on the first trajectory run the weight auto-downloads, checksummed and cached. Gemini-only users never trigger a weights download and, if they already have ffmpeg, need nothing else.",

"fit": "This fits the dual torch env constraint because download-on-demand weights plus system ffmpeg mean neither env, the main 2.6 cu124 or the WASB 1.11 cu113, needs freezing, avoiding two large PyInstaller bundles; the pip or installer plus the existing console script entry point and on-demand artifacts is the right shape, and the fetcher just sets the WASB_WEIGHTS env var for whichever env runs wasb_infer. On ffmpeg, the engine invokes bare ffmpeg and ffprobe in video.py and compiler.py and hard-needs ffprobe plus the libx264 encoder plus drawtext today, which a winget GPL build provides and which imageio-ffmpeg does not because it omits ffprobe, so a user-installed system ffmpeg is both the working and the license-clean choice. On GCS weights, the bytes already live in a GCS bucket and the engine already has fetch_to_local plus the WASB_WEIGHTS env override with env winning over config, so first-run download is a thin wrapper, and a public-read bucket avoids the storage-gcs extra and any credentials. For the FastAPI plus SPA split, binary and weight resolution is server-side and orthogonal to the cross-repo khelsutra web SPA; the planned capabilities endpoint should report whether ffmpeg is present and weights are cached so the UI only offers honestly runnable segmenters. For the non-tech India coach on Windows, winget is the lowest-friction install and doctor.py already prints the exact command, paired with the existing English, Kannada, and Hindi setup copy, a loud resumable first-run download fronted by a CDN for India latency, and a documented manual fallback for boxes where winget is disabled. The licensing guardrail holds because MIT weights are redistributable, ffmpeg stays user-installed so its GPL never touches the MIT bundle, and the only new packaging dependency, only if Hugging Face Hub is chosen, is Apache-2.0 huggingface_hub.",

"refuted": []

},

{

"topic": "gpu-cpu-cross-platform",

"recommendation": "Adopt a first-run LAUNCHER/BOOTSTRAP that owns hardware detection and installs the matching torch into the bundled venv (option C), implemented on top of uv `--torch-backend=auto` (option A) rather than hand-rolled --index-url branching. Concretely: (1) keep pyproject.toml as-is ? torch stays out of [project.dependencies] and the `gpu` extra stays docs-only (it's correct, do not try to express the index there); (2) the bootstrap runs `uv pip install torch torchvision --torch-backend=auto` into the MAIN env, which yields CUDA on a NVIDIA box, CPU elsewhere, and the MPS-capable PyPI wheel on macOS ? exactly mirroring what DeviceContext('auto') decides at runtime; (3) expose an explicit '--cpu' / device override in the launcher that maps straight to DeviceContext's cpu/cuda/mps so a user can force a backend (the faster-whisper/our-own contract); (4) treat the WASB torch-1.11 conda env as a SEPARATE bootstrap step the launcher drives by path (it cannot share the main env ? two incompatible torch versions), installing it only when the user picks the wasb_hybrid segmenter, and fetching wasb_badminton_best.pth.tar from GCS in the same step; (5) keep startup.py's best-effort warning (CPU_FALLBACK / CUDA_NOT_VISIBLE) loud in the web UI so we never repeat Ollama's silent-slow-CPU trap. Avoid option E (PyInstaller single bundle): the 4-5GB-vs-CPU split plus the two-torch-env reality make a single frozen artifact impractical and it contradicts the D10 'spin up a webserver' decision. Option D (CPU-default + documented GPU upgrade) is the safe fallback if shipping/embedding uv proves problematic, but it pushes a manual second step onto exactly the non-technical persona we're targeting.",

"fit": "Our codebase is already shaped for the run-time half of this problem:

DeviceContext (device.py:32-56) + the auto/cuda/cpu/mps contract + startup CPU_FALLBACK/CUDA_NOT_VISIBLE warnings (startup.py:104-145) replicate exactly what Ollama/faster-whisper do at run time. The GAP is purely install-time wheel selection, which pyproject.toml correctly refuses to express (torch out of deps, empty docs-only `gpu` extra, both --index-url recipes commented at pyproject.toml:58-65). The cleanest fit for the downloadable D10 'self-hosted webserver the user spins up' is a thin first-run launcher that runs `uv pip install torch --torch-backend=auto` into the bundled venv ? detection result and DeviceContext('auto') then agree by construction. Two app-specific complications the comparable apps mostly DON'T have: (a) the WASB native runner is a SECOND, incompatible torch env (torch 1.11+cu113 vs main 2.6+cu124) invoked by path with WASB_PYTHON/WASB_DEVICE/WASB_WEIGHTS (native_wasb_runner.py:80-136) ? the launcher must provision it as a separate optional step, never merge it into the main env; and (b) model weights live in GCS, so the same bootstrap should pull wasb_badminton_best.pth.tar. ffmpeg/ffprobe remain a system dep the launcher must check for (analogous to ComfyUI's external deps). Licensing is clean: uv (Apache-2.0/MIT) and pip-from-pytorch-index both satisfy the MIT/Apache/BSD-only guardrail; no new viral or training-data dependency is introduced. For the non-technical India-coach-on-Windows persona, the launcher-picks-the-wheel UX (Pinokio-style, option C) beats forcing them to learn --index-url, while preserving every existing CUJ because the engine entrypoints (indexer --server, python -m backend.worker) and device config flow are untouched.",

"refuted": []

"ComfyUI ships TWO distinct distributions: (a) ComfyUI Desktop (Comfy-Org/desktop) is an Electron app that bundles its own Python tooling and uses uv to install torch+deps at first run (backend resolved at install time, not pre-frozen); (b) ComfyUI Windows Portable is a separate non-Electron artifact that ships embedded Python (python_embedded) with per-GPU prebuilt torch wheels (one fixed backend per artifact). Both are valid precedents, but for different options: Desktop ~ the uv-bootstrap pattern (option A/C); Portable ~ the prebuilt-per-backend artifact pattern."

```
]
},
{
  "topic": "launcher-ux-wrapper",
  "recommendation": "Adopt (b)+(d): a tiny system-tray LAUNCHER that owns the server
lifecycle and auto-opens the browser, distributed by a one-click Inno Setup installer. This
SATISFIES D10's intent rather than violating it ? the rendered product stays the khelsutra SPA
in the user's own browser at 127.0.0.1:8000 (no embedded webview, no UI fork); the tray is only
a friendly on/off switch + status light + language toggle, i.e. exactly 'a self-hosted webserver
the user spins up' made non-scary. A launcher is NOT a desktop app in D10's sense; option (c)
(Electron/Tauri embedded-webview shell) IS, and it should be rejected: it contradicts D10's
letter (the superseded DESKTOP_APP_PLAN), adds a WebView2/Chromium runtime burden on varied
Indian Windows hardware, and collides head-on with the hard packaging problem because its
sidecar pattern presumes a PyInstaller freeze that blows up to 3-5GB with CUDA and can't host
our separate WASB conda env.\n\nSequencing (smallest-first, evidence-graduated, matching repo
norms): Slice 1 ? ship option (a) immediately: have the `indexer` console entry optionally
health-poll then webbrowser.open after --server (a few stdlib lines, zero new deps, zero license
risk) so the auto-open habit lands first. Slice 2 ? wrap it in the (b) pystray tray
(Open/Restart/View logs/Language/Quit, free-port fallback for the documented port-8000-busy
failure, crash balloon, graceful child-process kill), FIRST resolving the pystray LGPL-3.0
guardrail question (get a dynamic-link OK or swap to an MIT tray lib). Slice 3 ? the (d) Inno
Setup installer that provisions a managed env the Transformer-Lab way (isolated Miniconda/uv in
an app-specific dir for the dual-torch + WASB env, locate/ship LGPL ffmpeg, fetch WASB weights
from GCS on first run with a progress UI), drops a Start-Menu shortcut to the tray launcher, and
(optionally) a winget manifest. Budget a code-signing cert so SmartScreen/AV don't scare non-
tech users off the .exe.\n\nFirst-run wizard: put it IN the web UI (the Open WebUI/Ollama
precedent), launched on the tray app's first open, with 3 steps grounded in existing seams ? (1)
Gemini key: paste-or-skip, stored locally, with the consent copy that ?1280px chunks leave the
machine (never a training source per the guardrail); (2) Compute: pick Local GPU / Local CPU /
Cloud, rendered from GET /api/capabilities so it only offers what the box can honestly do
(truly-local vs cloud-serving presets) and disables options the hardware can't back; (3) Videos:
point at a local folder (or bucket/Drive later). Keep all three steps en/kn/hi-localized to stay
consistent with the trilingual SPA work, and emit structured logs at each step per the project's
launch-observability bar.",
```

```
"fit": "This engine is uniquely ill-suited to the 'freeze it into one .exe' instinct,
which is why the launcher+installer split is the right call. (1) Dual torch envs: native WASB
needs torch 1.11+cu113 in a SEPARATE conda env while the engine runs torch 2.6+cu124 ? a single
PyInstaller freeze cannot hold both, and PyInstaller+torch+CUDA is independently a 3-5GB 'CUDA
libs can't be split' dead-end; the Transformer Lab precedent (reject PyInstaller, ship an
isolated Miniconda that stays modifiable post-install) maps 1:1 onto our reality. (2)
torch/torchvision are deliberately NOT in pyproject deps because their CUDA/CPU wheels need
--index-url (PEP621 can't express it) ? so the INSTALLER, not a frozen bundle, is the only
honest place to resolve the right wheel for the user's GPU/CPU; ffmpeg stays a system dep (ship
an LGPL build) and WASB weights are pulled from the GCS bucket on first run. (3) The decoupled-
compute seam (M1-M8: deployment.profile/compute_target on AppConfig, PROFILE_PRESETS truly-
local/cloud-serving/cpu-mock, StorageBackend + ComputeBackend registries, startup.py per-profile
checks) already turns 'GPU-or-cloud' into a SETTING surfaced by GET /api/capabilities ? so the
wizard's compute step is config wiring, not net-new. (4) D10 + the live
LOCAL_APPLIANCE_ARCHITECTURE design already commit to browser-on-localhost (127.0.0.1:8000,
edge-gate/firewall as the tenancy boundary); a tray launcher preserves that origin model exactly
while an Electron/Tauri shell would fork it. (5) The SPA lives in khelsutra/web (Vite/React,
already en+kn+hi) ? the launcher/installer must serve THAT bundle (bundling-the-UI is the known
cross-repo concern), and the wizard/tray strings must stay trilingual to match. (6) Every added
dependency must clear the MIT/Apache/BSD guardrail: Inno Setup, Tauri, pywebview,
webbrowser(stdlib) are clean; pystray is LGPL-3.0 and is the one flag to resolve before shipping
option (b). Paid Gemini stays inference-only with explicit consent copy, never a training source
? the wizard's key step must say so.",
```

```
"refuted": [
```

"Compute placement maps onto existing engine CONFIG presets (backend/config/presets.py PROFILE_PRESETS: truly-local / cloud-serving / cpu-mock), but GET /api/capabilities does NOT exist ? it is explicitly net-new ([design]) per the appliance doc (lines 35/49/113). The wizard step needs a new endpoint to surface those presets to the UI."

```
]
},
{
  "topic": "prior-art-deepdive",
  "recommendation": "Adopt a layered, profile-matched strategy, not one installer. (1) FIRST, finish `native_wasb_runner.py` (the in-process torch path that retires the WSL/conda 'wasb' sidecar seen at wasb_runner.py:74,162). Almost every clean packaging story below collapses to ONE torch environment ? so removing the second incompatible env is the highest-leverage prerequisite, and it's already designed-for (base.py:46-54). (2) For the NON-TECH WINDOWS user, copy the Ollama+ComfyUI-desktop pattern: a small self-branded installer (license-clean Tauri shell, or even a plain NSIS/portable launcher ? NOT the GPL ComfyUI/Fooocus code) that bundles `uv`, on first run uses `uv` with `[[tool.uv.index]]`/`[tool.uv.sources]` to install torch/torchvision from the correct CUDA-or-CPU `--index-url` (the precise thing pyproject.toml says PEP 621 can't do), bundles an LGPL ffmpeg build (openh264/NVENC, no GPL x264 per D4), downloads the GCS WASB weights on first run with a progress bar (Ollama/GPT4All model-hub UX), then launches `python -m backend.main --server` and opens 127.0.0.1:8000 to the existing khelsutra React SPA. Update like Ollama: run a new installer over the old, weights preserved. (3) For the SERVER / cloud-serving profile, build the Frigate-style Docker image (bundled ffmpeg + per-accelerator variant) ? which ALSO closes the 'no Dockerfile' gap blocking Cloud Run+GPU. (4) As a near-zero-effort additional channel, publish a Pinokio install script (MIT, app-isolated envs + ffmpeg + GPU libs out of the box) so technically-curious users get one-click install without us owning a launcher. Do NOT make Docker-on-Windows the non-tech path: Frigate itself shows Docker+GPU passthrough on Windows is exactly what that user cannot debug.",
  "fit": "Our app is unusual vs the LLM crowd (Ollama/LM Studio/Jan/GPT4All): they bundle ONE static C++ engine (llama.cpp) so 'Python' disappears, whereas we ship a Python+FastAPI+torch CV pipeline + ffmpeg + cloud-fallback. So the LLM apps teach UX (GUI weight-hub, headless-daemon-behind-OpenAI-server, runtime versioning) but not the core mechanic. The directly-applicable mechanics come from the SD/diffusion + Frigate camp: (a) `uv` is the one tool that resolves the torch `--index-url` blocker our own pyproject.toml:8-11 documents as un-expressible ? adopt it. (b) Pinokio (MIT) and ComfyUI-portable already run multiple isolated torch/CUDA envs side by side, which is the literal shape of our wasb_runner.py conda('wasb')-vs-engine split ? but the cleaner long-term move is to finish native_wasb_runner.py so there's ONE torch. (c) Frigate is our closest functional analog (ffmpeg+ML+web UI) and gives the bundled-ffmpeg-inside-the-artifact + per-accelerator-variant + Docker-tag-update patterns, AND building that image closes our verified 'no Dockerfile' gap (LOCAL_APPLIANCE_ARCHITECTURE.md:49,96) for Cloud Run+GPU. The delivery SHAPE must honor owner D10 (self-hosted webserver+UI the user spins up, not a desktop app, compute/storage as settings) and the dual local/cloud requirement ? which is why the recommendation splits into a non-tech Windows installer profile and a Docker server profile rather than one artifact. Licensing guardrail forces shell choice: ComfyUI-desktop/Fooocus are GPL-3.0 and Pinokio is MIT ? copy GPL patterns but only vendor MIT/Apache/BSD (Tauri shell, uv, LGPL ffmpeg, MIT WASB weights). Engine reuse is total: a thin shell just spawns the existing `python -m backend.main --server` / `python -m backend.worker {infer|train}` and the existing presets.py truly-local/cloud-serving profiles select compute.",
  "refuted": [
    "A Cloud Run GPU worker Dockerfile now EXISTS (deploy/cloudrun/Dockerfile, shipped 2026-06-21 in #287/M6); it bundles ffmpeg and builds the WASB torch-1.11.0+cu113 conda env. The khelsutra 'no Dockerfile' claim is stale (the doc predates the Dockerfile by ~1 hour). The 'WASB env makes containerization non-trivial' point is still valid ? the new image confirms it by keeping two separate python stacks ? but it is no longer an unaddressed blocker."
  ]
},
{
  "topic": "cloud-one-click",
  "recommendation": "Make ONE artifact, TWO official run modes, and resist hosted-PaaS GPU buttons. (1) THE SPINE - already built (M1-M8, default-OFF): ship one pip package (indexer console script) + one worker Docker image (deploy/cloudrun/Dockerfile), and let deployment.profile pick everything. backend/compute/__init__.py:47-49 already makes local<->cloud a single config switch (cloud_run->CloudRunCompute, else inline); the dispatched Cloud Run Job runs the SAME image forced inline (cloud_run.py:41-44). Textbook twelve-factor; nothing new is needed to claim 'local AND cloud from one artifact' - only finishing M7 (real L4 run) and wiring worker usage-emission into the M8 ledger. (2) DOWNLOADABLE-APP 'push my workload
```

to cloud' MODE: in the self-hosted UI Settings, surface profile as a toggle that calls suggest_profile() to PROPOSE a default but requires explicit confirm (D15 - cloud spend never silent). 'Push to cloud' = flip to cloud-serving (compute_target=cloud_run + storage=gcs); the local control plane then dispatches GPU detect+stitch to a Cloud Run Job and bucket-polls. Show the live per-video cost estimate (M8, INR) BEFORE confirming, and a hard per-video cap (open owner question in LOCAL_APPLIANCE_ARCHITECTURE section 10). (3) CLOUD TARGET = Cloud Run GPU Jobs, NOT Render/Railway/Fly for GPU: Render and Railway have no GPU; Fly's GPUs are deprecated after Aug 1. Cloud Run L4 (~\$0.67/hr, per-second, scale-to-zero, ~5s GPU-ready) is the only option that ALSO matches per-video billing and ADR-013 (GCP-only). Keep a 'Deploy to Render' button as a SECONDARY CPU-only control-plane distribution (Gemini/motion/stub modes that dispatch GPU work back to GCP) - lowest-friction one-click for a non-technical coach, but never the GPU path. (4) BYO-cloud: keep create_gpu_vm.sh/teardown.sh as the warm no-cold-start path for power users/owner; surface the always-DELETE rule in a UI teardown affordance. (5) NON-NEGOTIABLE before any public exposure: Cloudflare Access (cloudflared tunnel, no inbound ports) + PyJWT verification of the Cf-Access JWT signature (aud + JWKS refreshed for the ~6-week rotation) + Cloud Run ingress=internal-and-cloud-load-balancing - both header-trust AND signature-verify, because header-only trust is spoofable (D9). (6) NO DATA LEAK: truly-local needs no DPA; cloud-serving must honor D10 (adults-only training, derived-data, raw auto-delete, paid=no-train) and a retention sweep (originals are never deleted today) - all counsel-gated to M9 under DPDP/GDPR. Sequence: finish M7 (real L4 + DEPLOY_REPORT) -> land M9 edge-auth (default-OFF) -> wire ledger usage-emission + per-video cap -> add the UI profile toggle with cost-confirm -> optional Render control-plane button last.",

"fit": "Maps onto YOUR engine precisely. (a) torch dual-env: deploy/cloudrun/Dockerfile ALREADY encodes the two-stack problem - a main app env (torch 2.6+cu124-class) plus a separate conda 'wasb' env (py3.8 torch 1.11+cu113) that the native runner shells out to via \$WASB_PYTHON - so the cloud path inherits the exact split the local GPU box uses (mirrors deploy/digitalocean/gpu_setup.sh); for the DOWNLOADABLE app the same split is why GPU mode needs a bundled image or guided installer rather than pure pip (torch/torchvision deliberately NOT pip deps - need --index-url). (b) ffmpeg: a system dep, installed in the image (ffmpeg + DejaVu fonts for the watermark) and required by the local console; stitch runs co-located with detect on Cloud Run (D4/D16) so its CPU/wall/egress meter into one job. (c) GCS weights: wasb_badminton_best.pth.tar lives in the bucket and is staged at runtime via \$WASB_WEIGHTS (WASB-C3 - uncleared commercial provenance, so NEVER baked in and resolved before public serving); the StorageBackend gcs registry + deploy/gcp/provision.sh already provision the bucket+SA. (d) FastAPI+SPA: the engine FastAPI loop is reused unchanged behind the gate; CORS '*' + 127.0.0.1 bind (main.py:51,735) means the appliance is a reverse-proxied single origin (khelsutra LOCAL_APPLIANCE_ARCHITECTURE D1/D2) and the SPA (khelsutra/web, en+kn+hi) is bundled cross-repo - the 'push to cloud' toggle is one Settings screen rendered from /api/capabilities. (e) non-tech India coach on Windows: today's verified real-GPU path is owner Windows+WSL2 (detector_impl=auto); for a coach, 'push my workload to cloud' lets them avoid GPU setup entirely - confirm-first, INR cost shown, Cloudflare Access email-OTP login (free <=50 users), no inbound ports on their network. Every new packaging dependency stays MIT/Apache/BSD; Gemini stays inference-only never a training source; DPDP under-18 = reel=yes/training-no (D10).",

"refuted": []

}

}

}

CONTEXT:

PROJECT: a local, AI-powered badminton/racket-sports HIGHLIGHT INDEXER + rally detector. Engine repo (PRIMARY): C:/Users/avidu/Projects/badminton-highlight-indexer (Python, FastAPI + SQLite + ffmpeg + GPU CV/AI; paid Gemini = cloud AI). Sibling repos: C:/Users/avidu/Projects/khelsutra (the web/ React SPA + site/ marketing), C:/Users/avidu/Projects/rally-annotator (VLC plugin, MIT), C:/Users/avidu/Projects/sports-data-collector, C:/Users/avidu/Projects/sports-obsreport.

GOAL of the plan we are building: bake the engine into a DOWNLOADABLE, "batteries-included" Python APP that a user can download and run WITHOUT sourcing the repo (no git clone / build-from-source). It must spin up a local WEB SERVICE that runs (a) INFERENCE (video -> rallies -> highlight reel), (b) TRAINING (BYO model on their own labelled data), and (c) RALLY ANNOTATION if needed -- runnable BOTH locally and against CLOUD services. Preserve every already-completed end-to-end CUJ.

KEY FACTS the orchestrator already verified:

- pyproject.toml: hatchling build, namespace package (no backend/__init__.py), packages=["backend","training"], console script: indexer = backend.main:main. requires-python >=3.10.
- torch/torchvision are DELIBERATELY NOT in dependencies -- their CUDA/CPU wheels need --index-url (PEP621 cannot express). Optional extras: gpu(empty, docs-only), auth(PyJWT), reporting(git+ssh obsreport), storage-s3(boto3), storage-gcs(google-cloud-storage). google-genai is a hard dep.
- Server today: python -m backend.main --server --port 8000 . Worker: python -m backend.worker {infer|train} . CLI single-shot: python -m backend.main --video-path ... --segmenter {gemini|yolo_hybrid|wasb_hybrid|motion}.
- THE HARD PACKAGING PROBLEM: native WASB shuttle model runs in a SEPARATE Python env (conda, torch 1.11+cu113) while the main engine uses torch 2.6+cu124. ffmpeg/ffprobe are system deps. Model weights (wasb_badminton_best.pth.tar) live in a GCS bucket.
- Decoupled-compute SEAM already built (M1-M8 merged, default-OFF): deployment.profile + compute_target on AppConfig; backend/config/presets.py (PROFILE_PRESETS: truly-local / cloud-serving / cpu-mock); backend/storage/* (local/gcs/s3/gdrive StorageBackend registry); backend/compute/* (ComputeBackend registry, CloudRunCompute dispatches a Cloud Run Job); backend/config/startup.py per-profile checks; backend/billing.py cost ledger.
- The web SPA lives in a DIFFERENT repo (kheksutra/web, Vite/React/TS/Tailwind, i18n en+kn+hi). The engine has a likely-DEAD backend/api/static. Bundling the UI is a cross-repo concern.
- Licensing guardrail (NON-NEGOTIABLE): every dependency -- code, data, AND weights -- must be MIT/Apache-2.0/BSD only. Paid LLMs = inference/serving only, never training data. Any NEW packaging dependency must also be license-clean.
- A prior docs/DESKTOP_APP_PLAN.md exists but was SUPERSEDED by owner decision D10: deliver a "self-hosted webserver + UI the user spins up" (NOT a desktop app). kheksutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md is the live appliance design.

Always ground claims in file:line or a doc path. Read the actual files; do not assume.

2. assistant (2026-06-21T14:05:23.098Z)

I'll examine the key files referenced in the digest to ground my critique before producing findings.

3. user (2026-06-21T14:05:24.643Z)

7835f34 feat(gcp): always-install Ops Agent on GPU VMs + a cloud-GPU dry-run guide (#291)
aac7633 feat(serving): M9-auth ? Cloudflare Access edge-auth + per-tenant owner_id (default-OFF) (#290)
e68b583 fix(api): resolve a bucket-ingested video's source to a local file before stitching (#289)

- ABLATION_LAYER.md
- ANALYZERS_AND_RECONCILERS.md
- BACKLOG.md
- CLOUD_GPU_DRYRUN_GUIDE.md
- CODE_AUDIT_AND_TEST_HARDENING.md
- CODE_MAP.md
- COMMERCIALIZATION.md
- COMPUTE_DECOUPLED_SERVING
- DEFERRED_HARDENING_PLAN.md
- DESKTOP_APP_PLAN.md
- DOC_STATUS.md
- EVALUATION.md
- EXECUTION_POLICY.md
- EXPERIMENT_HARNESS.md
- GOLDEN_REGRESSION_FIXTURES.md
- HARDENING_LOOP_HANDOFF.md
- HOSTING_PLAN.md
- HOW_RALLY_DETECTION_WORKS.md
- I18N_PLAN.md
- KHEKSUTRA_PRODUCT_OVERVIEW.md
- MULTI_SIGNAL_FUSION_PLAN.md
- NEXT_STEPS.md
- OWNED_MODEL_IMPLEMENTATION_PLAN.md
- PERSONALIZATION_PLAN.md

PER_VIDEO_WORKSPACE.md
PLATFORM_ARCHITECTURE.md
POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md
PROJECT_DOC_TEMPLATE.md
PROMINENT_COURT_DETECTION.md
QUALITY_ITERATIONS.md
R1_MILESTONE_LAUNCH_REPORT.md
R2_EVAL_METRIC_DESIGN.md
RALLY_DETECTION_GAP_CLOSURE.md
RALLY_DETECTION_QUALITY_REPORT.md
RALLY_QUALITY_RESEARCH.md
README.md
REAL_FOOTAGE_VALIDATION_REPORT.md
ROADMAP.md
SETUP_NEW_MACHINE.md
STORAGE_SHARING_MODEL.md

4. assistant (2026-06-21T14:05:27.438Z)

Let me read the key files to ground my critique.

5. user (2026-06-21T14:05:29.064Z)

PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).

Build backend: hatchling. `backend/` is an intentional NAMESPACE package (no `backend/\_\_\_init\_\_.py`; mypy.ini sets explicit_package_bases for the same reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's auto-detection would otherwise miss the un-___init___'d `backend` root.

torch/torchvision are deliberately NOT listed in [project.dependencies]: their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url), which PEP 621 cannot express. Install them separately ? see [project.optional-dependencies] for `gpu` (documentation only) and the README. CI installs CPU torch on its own line.

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

```
[project]
name = "badminton-highlight-indexer"
version = "0.1.0"
description = "AI-assisted rally segmentation and highlight indexing for racket sports."
readme = "README.md"
requires-python = ">=3.10"
license = { text = "MIT" }
authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
```

Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).

```
dependencies = [
    "fastapi>=0.111.0",
    "uvicorn>=0.30.1",
    "opencv-python>=4.9.0.80",
    "numpy>=1.26.4",
    "pydantic>=2.7.1",
    "jinja2>=3.1.4",
    "python-dotenv>=1.0.0",
```

```

    "google-genai>=2.4.0",
    "supervision>=0.21.0,<0.30.0",
]

```

```
[project.optional-dependencies]
```

Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).

```

dev = [
    "pytest",
    "pytest-cov",
    "httpx",          # fastapi TestClient transport
    "ruff",
    "mypy",
    "PyJWT[crypto]>=2.8", # M9 edge-auth: Cf-Access JWT verify (RS256) ? tests + CI
]

```

M9 edge-auth (Cloudflare Access). DEFAULT-OFF: only needed when security.edge_auth_jwt is lazy-imported, so the base runtime works without this; install it for hosted/multi

```
auth = ["PyJWT[crypto]>=2.8"]
```

Per-video observability reports (SOFT dependency: the pipeline degrades to a no-op if absent). Pinned to its git tag per BACKLOG. The private repo is fetched over SSH; if you lack access, simply omit this group ? reporting tests skip via pytest.importorskip. For local dev against a sibling checkout:

pip install -e ../sports-obsreport --no-build-isolation

```

reporting = [
    "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
]

```

`gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot express, so they are NOT declarable here. Install them out-of-band instead:

CUDA 12.4 box:

pip install torch>=2.12.0 torchvision>=0.27.0 --index-url https://download.pytorch.org

CPU-only box (also what CI uses):

pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu

```
gpu = []
```

Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs these. Cloud SDKs are imported lazily, so `import backend.storage` works without the install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare R2 DO Spaces + MinIO via endpoint_url.) `gdrive://` needs NO extra ? it reads a locally MOUNTED Google Drive (Google Drive for Desktop) as plain files; see backend/storag

```

storage-s3 = ["boto3>=1.34"]
storage-gcs = ["google-cloud-storage>=2.16"]

```

```
[project.scripts]
```

`indexer --server`, `indexer --help`, etc. -> backend/main.py::main()

```
indexer = "backend.main:main"
```

```
[tool.hatch.metadata]
```

The `reporting` extra pins obsreport to a git tag (a PEP 508 **direct reference); hatchling requires opting in to allow those in metadata.**

```
allow-direct-references = true
```

```
[tool.hatch.build.targets.wheel]
```

`backend` is a namespace package (no `__init__.py`); list both import roots explicitly so an editable/wheel install exposes `import backend` and import training` regardless of cwd.`

```
packages = ["backend", "training"]
```

6. user (2026-06-21T14:05:29.872Z)

```
from backend.reporting import emit_indexer_report
from backend import storage # M2: URI-aware input acquisition (local = identity passthrough)
from backend import billing # M8: per-job cost ledger (USD internal / INR display)
from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-OFF)
```

```
app = FastAPI(title="Sports Highlight Indexer API")
```

```
def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
    """M9: CORS allow-origins from the `RALLY_CORS_ALLOW_ORIGINS` env var (comma-separated),
    default `["*"]`. Read from the ENV (not a cwd `config.json`) so importing
    `backend.main`
    does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to its
    origin."""
    src = os.environ if env is None else env
    raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
    origins = [o.strip() for o in raw.split(",") if o.strip()]
    return origins or ["*"]
```

CORS for the web UI. `allow_origins='*' with allow_credentials=True is rejected by browser fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so credentials off. The origin list is configurable via the env var above (default ["*"]) so a hosted deployment lock it to its origin; the middleware is fixed at startup.`

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=_cors_origins_from_env(),
    allow_credentials=False,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Serves static frontend files directly (now under `api/` per approved structure)

```
os.makedirs("backend/api/static", exist_ok=True)
app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
```

```
@app.get("/", response_class=HTMLResponse)
def serve_index():
    index_path = "backend/api/static/index.html"
    if os.path.exists(index_path):
        with open(index_path, "r", encoding="utf-8") as f:
            return f.read()
    return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
```


Global variables initialized at startup

```
db_instance: Optional[Database] = None
global_config: Optional[AppConfig] = None # now typed (Pydantic model)
```

Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION now lives in backend.api.job_worker. Re-exported here so the names stay addressable in `backend.main` (`JobWaiting`, `\_drain\_due\_jobs`, `\_arm\_wake\_timer`, `\_get\_executor`, `\_run\_claimed\_job`, `\_MAX\_DRAIN\_WAKE\_SEC`, ...)). The worker resolves these seams in this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still

```
=== END ===
    else:
        threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
        print(f" Sample Frame Count: {len(motions)}")
        print(f" Average motion index around timestamp: {round(avg_motion, 4)} (threshold:
{threshold})")
        if avg_motion > threshold:
            print(" Result: [ACTIVE PLAY] High motion detected. Segment classifies as active
play.")
        else:
            print(" Result: [DEAD TIME] Idle movement. Segment classified as dead time.")
    else:
        print(" [ERROR] No visual frames were extracted in the sample range.")

    print("=" * 60)

def _enforce_startup_or_exit(config) -> None:
    """M5: run the per-profile startup checks for the server/CLI path and ``sys.exit(1)`` on a
    FATAL incoherent profile (e.g. cloud-serving pointed at local-only storage) BEFORE any work.
    A strict no-op when no ``deployment.profile`` is selected (today's behaviour). No GPU probe
    here ? the server stays torch-light; the detector healthcheck is the GPU authority.
    Extracted
    so the fail-fast wiring + the default-OFF no-op are unit-testable."""
    try:
        enforce_startup_checks(config, logger=logger)
    except StartupCheckError as e:
        logger.error("[STARTUP] refusing to start: %s", e)
        sys.exit(1)

def main():
    parser = argparse.ArgumentParser(description="Sports Highlight Indexer Orchestrator")
    parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
    parser.add_argument("--output-dir", default="output", help="Directory where processed
clips/highlights are saved")
    parser.add_argument("--setup", action="store_true", help="Launch dependency checker and
setup utility")
    parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a video clip
to run detailed diagnostics on")
    parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
    parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI server
on")
    parser.add_argument("--max-frames", type=int, help="Limit processing to the first N sub-
sampled frames for debugging")

    available_segmenters = list(segmenter_registry.list_available().keys())
    parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
    parser.add_argument("--trim-start", type=float, help="Start time in seconds for the debug
```

```

trim segment feature")
    parser.add_argument("--trim-end", type=float, help="End time in seconds for the debug trim
segment feature")
    parser.add_argument("--trim-output", help="Output filename for the debug trimmed segment
(saves in output-dir unless absolute path)")

    args = parser.parse_args()

    if args.setup:
        # Invoke the diagnostics/bootstrap script (renamed from the old root
        # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
        # build system). Resolve its path from this file so `indexer --setup`
        # works regardless of the caller's cwd.
        import subprocess
        doctor_path = os.path.join(
            os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
            "scripts", "doctor.py",
        )
        print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
        subprocess.run([sys.executable, doctor_path])
        return

    if args.trim_start is not None and args.trim_end is not None:
        if not args.video_path:
            print("[ERROR] Please provide --video-path to extract a segment.")
            return

        # Determine output path
        out_filename = args.trim_output or "trimmed_segment.mp4"
        if os.path.isabs(out_filename):
            out_path = out_filename
            out_dir = os.path.dirname(out_path)
            out_name = os.path.basename(out_path)
        else:
            out_dir = args.output_dir
            out_path = os.path.join(out_dir, out_filename)
            out_name = out_filename

        os.makedirs(out_dir, exist_ok=True)
        print(f"[CLI] Extracting debug segment from {args.trim_start}s to {args.trim_end}s from
{args.video_path}...")

        # global_config isn't initialized this early; load the typed config so the trim
        # clip carries the same configured stitching paddings + watermark as a real
        # highlight (watermark default-on).
        stitching = load_app_config().stitching
        compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
                                end_padding=stitching.end_padding,
                                watermark=stitching.watermark)
        try:
            res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
            print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
        except Exception as e:
            print(f"[ERROR] Failed to compile trimmed segment: {e}")
        return

    if args.debug_clip is not None:
        if not args.video_path:
            print("[ERROR] Please provide --video-path to debug a specific clip.")
            return
        run_cli_diagnose_clip(args.video_path, args.debug_clip)
        return

    # Initialize Global Config and DB

```

```

global global_config, db_instance
global_config = load_app_config() # returns typed AppConfig
_enforce_startup_or_exit(global_config) # M5: fail fast on an incoherent ACTIVE profile

# The CLI --output-dir overrides the configured output directory. It now lives
# on the typed model (OutputConfig.output_dir), so every consumer ? including
# /api/compile ? reads the same value instead of a write-only runtime hack.
global_config.output.output_dir = args.output_dir
os.makedirs(global_config.output.output_dir, exist_ok=True)

db_path = os.path.join(global_config.output.output_dir, global_config.output.db_name)
db_instance = Database(db_path)

# Crash recovery (#16): jobs left queued/running by a previous process are dead ?
# the executor is in-process. Mark them failed so pollers see a terminal state.
swept = db_instance.fail_stale_jobs()
if swept:
    logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ? failed.")

# CLI processing mode (no web UI needed)
if args.video_path and not args.server:
    print(f"[CLI] Processing video file: {args.video_path}")
    # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is fetched to
scratch.
    # The local-existence fast-fail only applies to a local input (stat the RESOLVED key,
not the
    # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error, not a
crash.
    storage_cfg = getattr(global_config, "storage", None)
    try:
        input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
    except ValueError as e:
        print(f"[FAIL] {e}")
        return
    if input_ref.backend == "local" and not os.path.exists(input_ref.key):
        print(f"[FAIL] Video file not found: {args.video_path}")
        return
    local_video = storage.acquire_local_input(args.video_path, storage_cfg)

    video_id = compute_video_id(local_video)
    was_reingest = db_instance.get_video(video_id) is not None

    # Validate (with-context variant surfaces ffprobe_meta for the report)
    print("[CLI] Validating...")
    val_results, val_context = registry.run_all_with_context(local_video, global_config)
    failures = [r for r in val_results if not r.passed]

    # Save video status
    status = "failed" if failures else "processed"
    db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r in
val_results])

    seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
    segmenter_actual = None
    segmentation_ran = False
    try:
        print("\n" + "="*40)
        print("VALIDATION SUMMARY")
        print("="*40)
        for r in val_results:
            status_symbol = "[PASS]" if r.passed else "[FAIL]"
            print(f"{status_symbol} {r.validator_name}: {r.message}")
        print("="*40 + "\n")

```

```

if failures:
    print("[FAIL] Video failed checks. Indexing halted.")
    return

# Index
segmenter_cls = segmenter_registry.get(seg_name)
if not segmenter_cls:
    print(f"[FAIL] Segmenter '{seg_name}' not found.")
    return

print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
segmenter_actual = seg_name
play_wm, cand_wm = db_instance.segment_watermarks(video_id)
segmenter = segmenter_cls(db_instance, global_config)
result = segmenter.process_video(video_id, local_video, max_frames=args.max_frames)
segmentation_ran = True

if isinstance(result, Failure):
    print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
    print("[FAIL] Indexing halted for video.")
    segments = []
else:
    segments = result.value if hasattr(result, "value") else result
    print(f"[SUCCESS] Process complete. Indexed {len(segments)} play segments inside
database: {db_path}")
    # Destroy-AFTER-confirm: keep new on success, restore prior on empty/failure.
    _finalize_reingest(video_id, segments, play_wm, cand_wm)
finally:
    # Always emit the per-video observability report. Never raises.
    paths = emit_indexer_report(
        db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
        val_results=val_results, val_context=val_context,
        segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,

```

7. assistant (2026-06-21T14:05:33.623Z)

The digest's claims about main.py CWD-relative static mount and the entrypoint structure are confirmed. Let me check the config loader, presets, startup, the appliance doc, doctor.py, and whether net-new endpoints exist.

8. user (2026-06-21T14:05:35.412Z)

"""Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING M1).

A *deployment profile* is a single switch that applies a coherent BUNDLE of existing config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one knob at a time (the design in ``docs/COMPUTE\_DECOUPLED\_SERVING/README.md`` §4.3).

Precedence (lowest ? highest), realised by the loader::

```
built-in model defaults < profile preset < config.json < env < worker --set
```

This module is *pure data + pure functions* ? no IO, and (critically) *no torch import at module load*. The loader applies a preset *ONLY* when a profile is *EXPLICITLY* selected (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT\_PROFILE`` env var); with no profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` == today). Auto-detect only ever *suggests* a profile ? it never auto-applies one (D15: cloud spend is never entered silently).

```
from __future__ import annotations
```

```
import copy
import os
```

```
from typing import Any, Mapping, Optional
```

Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal`` these and ``tests/test\_config\_deployment.py`` pins parity so the two can never drift.

```
PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
```

Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to when config.json explicitly overrides compute_target into a state inconsistent with the

```
COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
    "truly-local": "local_gpu",
    "cloud-serving": "cloud_run",
    "cpu-mock": "cpu_mock",
}
```

A preset is a partial, nested override dict over the model defaults. It touches ONLY known that EXIST today ? M1 is pure/additive: the new ``deployment.{profile,compute\_target}``, ``indexing.{detector\_impl,skip\_ai\_handoff}`` and ``storage.backend``. Later milestones these bundles ? ``input\_backend`` (M2) and the configurable output/workspace base (Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.

```
PROFILE_PRESETS: dict[str, dict[str, Any]] = {
    "truly-local": {
        "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
        # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU box).
        # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
        "indexing": {"skip_ai_handoff": True},
        "storage": {"backend": "local"},
    },
    "cloud-serving": {
        "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
        # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor (D11).
        "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
        "storage": {"backend": "gcs"},
    },
    "cpu-mock": {
        "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
        # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression profile.
        "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
        "storage": {"backend": "local"},
    },
}
```

```
def is_known_profile(profile: str) -> bool:
    """True for a non-empty, recognised profile name."""
    return bool(profile) and profile in PROFILE_PRESETS
```

```
def preset_for(profile: str) -> dict[str, Any]:
    """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate the
    module-level table), or ``{}`` for ``""`` / an unknown profile."""
    return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
```

```
=== LOADER ===
```

```
from .presets import (
    COMPUTE_TARGET_FOR_PROFILE,
    PROFILE_PRESETS,
    is_known_profile,
```

```

    preset_for,
    suggest_profile,
)

logger = logging.getLogger(__name__)

DEFAULT_CONFIG_PATH = Path("config.json")

def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
    """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge, scalars
    replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
    overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-profile
    path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""
    out = dict(base)
    for key, val in override.items():
        if isinstance(val, dict) and isinstance(out.get(key), dict):
            out[key] = _deep_merge(out[key], val)
        else:
            out[key] = val
    return out

def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
    """The EXPLICITLY chosen deployment profile, env winning over config.json. ``""`` if none
    is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so a bare
    environment never silently selects a profile (and never silently spends on cloud).

    An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the file's
    profile (rather than suppressing it) ? so an env typo can't leave the recorded profile
    asserting a preset that was never applied. A bad value in config.json is left to surface as
    a hard, typed-Literal error downstream (config-file correctness)."""
    env_profile = os.environ.get("DEPLOYMENT_PROFILE")
    if env_profile and env_profile.strip():
        ep = env_profile.strip()
        if is_known_profile(ep):
            return ep
        logger.warning(
            "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring it "
            "and falling back to config.json deployment.profile.", ep, ",
".join(PROFILE_PRESETS),
        )
        # fall through to the file's profile (env typo must not suppress a valid file choice)
    dep = file_data.get("deployment")
    if isinstance(dep, dict):
        prof = dep.get("profile")
        if isinstance(prof, str) and prof.strip():
            return prof.strip()
    return ""

def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str = "") ->
list[str]:
    """Dotted paths in `data` that the typed config will not honor.
```

9. user (2026-06-21T14:05:35.695Z)

```

import sys
import os
import subprocess
import shutil
```

This diagnostics/bootstrap script lives in scripts/ but operates on the repo root (requirements.txt, config.json). Resolve the root from this file's

location so it works regardless of the caller's cwd.

```
REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

def check_command(cmd):
    """Check if a system command is available."""
    return shutil.which(cmd) is not None

def run_pip_install():
    """Install required Python packages."""
    print("Installing Python packages from requirements.txt...")

    # Check if NVIDIA GPU is present
    has_gpu = check_command("nvidia-smi")

    requirements_path = os.path.join(REPO_ROOT, "requirements.txt")
    pip_cmd = [sys.executable, "-m", "pip", "install", "--user", "-r", requirements_path]

    import platform
    if has_gpu:
        print("[INFO] NVIDIA GPU detected. Configuring PyTorch to download CUDA-enabled wheels.")
        pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cu124"])
    elif platform.system() == "Darwin":
        # macOS has no CUDA; the default PyPI wheel already ships CPU + Apple MPS.
        print("[INFO] macOS detected. Using default PyTorch wheels (CPU + Apple MPS).")
    else:
        print("[INFO] No NVIDIA GPU detected. Defaulting to PyTorch CPU wheels.")
        pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cpu"])

    try:
        subprocess.check_call(pip_cmd)
        print("[SUCCESS] Python packages installed.")
        return True
    except subprocess.CalledProcessError as e:
        print(f"[ERROR] Failed to install Python packages: {e}")
        return False

def init_default_config():
    """Initialize a default config.json if it doesn't exist.

    Now delegates to the canonical backend.config loader so that defaults
    stay in sync with the Pydantic models (single source of truth).
    """
    try:
        from backend.config import save_default_config
    except Exception as e:
        print(f"[ERROR] Could not import backend.config loader: {e}")
        print("[INFO] Falling back to no-op for config initialization.")
        return

    config_path = os.path.join(REPO_ROOT, "config.json")
    if os.path.exists(config_path):
        print(f"[INFO] {config_path} already exists. Skipping initialization.")
        return
```

10. user (2026-06-21T14:05:36.826Z)

No matches found

11. assistant (2026-06-21T14:05:40.356Z)

The endpoints aren't decorated with `@app.` in backend/api. Let me find where routes are actually registered, and check the appliance doc for the net-new endpoint claims plus the worker train flow.

12. user (2026-06-21T14:05:42.382Z)

```
Found 5 files
backend\main.py
backend\infrastructure\database.py
backend\config\models.py
backend\api\uploads.py
backend\api\static\app.js
```

13. user (2026-06-21T14:05:43.594Z)

```
71:os.makedirs("backend/api/static", exist_ok=True)
72:app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
74:@app.get("/", response_class=HTMLResponse)
75:def serve_index():
76:    index_path = "backend/api/static/index.html"
80:    return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
144:@app.get("/api/config")
148:@app.get("/api/health")
164:    This is the former synchronous /api/process body, unchanged in behavior, now run on
281:@app.post("/api/process", status_code=202)
283:    """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
328:        content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
332:@app.get("/api/jobs/{job_id}/cost")
347:@app.get("/api/videos/{video_id}/cost")
358:@app.get("/api/jobs/{job_id}")
386:app.include_router(uploads_api.router)
391:@app.get("/api/highlights/{video_id}/{filename}")
393:    """Stream a compiled highlight reel ? `filename` is the bare name given to /api/compile
410:        detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
415:@app.post("/api/query")
426:@app.post("/api/compile")
478:    # video ingested via /api/process from a URI, or a plain local path. Resolve it to a
LOCAL file
486:    download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
655:    # /api/compile ? reads the same value instead of a write-only runtime hack.
```

14. assistant (2026-06-21T14:05:48.271Z)

The digest claims `/api/capabilities`, `/api/videos` (list), `/api/reels`, and PATCH
`/api/segments` are net-new (missing). Confirmed: only `/api/process`, `/api/jobs`,
`/api/query`, `/api/compile`, `/api/highlights`, `/api/config`, `/api/health`,
`/api/videos/{id}/cost`, uploads router exist. No `/api/capabilities`, no segment-correction
PATCH, no reels list. Let me verify the appliance doc's net-new endpoint list and the worker
train gating, plus the bind address claim.

15. user (2026-06-21T14:05:50.760Z)

```
756:    uvicorn.run(app, host="127.0.0.1", port=args.port)
=== APPLIANCE DOC capabilities/endpoints ===
35:| D6 | Compute placement is a **setting**, rendered from `GET /api/capabilities` |
`[design]`; 2026-06-20 | UI offers ONLY what the box can honestly do |
49:- **Absent (net-new):** `GET /api/capabilities`, `GET /api/videos` (only single-row
`get_video(id)`), `GET /api/reels/{video_id}`, any source/proxy streaming endpoint, any
**Dockerfile**, any per-video **cost ledger** `[verified ? grep returns none]`.
87:| **Bucket URI** (`s3://`, `gs://`) | **POST /api/process` accepts the bucket URI directly**
(resolve ? fetch to scratch ? same pipeline ? API DB) `[verified main.py:159,274,283 ? corrected
2026-06-21; was "net-new"]`; the worker CLI also fetches `--video-uri`. So the **engine endpoint
is NOT net-new** ? only the SPA call that POSTs the URI + polls `/api/jobs` is. *(Default
single-user mode; HOSTED mode w/ `allowed_video_root` correctly rejects a bucket URI 400.)* |
**engine: DONE (verified); SPA ingest call: design (= B-P2)** |
90:### 4.3 Compute placement matrix (one UI, surfaced from `/api/capabilities`)
```



```

96:| **GPU host ? native WASB** (in-flight) | `wasb`+`detector_impl=native` ? `NativeWasbRunner`
(in-proc torch) | a **GPU host ? DO GPU droplet / GCE GPU VM / Cloud Run+GPU**; `WASB_WEIGHTS` |
**REAL**, on-device | partially wired (fusion builds it, trajectory refuses); **owner byte-
parity gate**; **Cloud Run/GCE need a containerized worker ? no Dockerfile exists, and WASB's
torch-1.11 conda env makes the image non-trivial** |
98:| **Remote GPU worker via bucket** (reserved) | console CPU + `storage=s3`; separate GPU host
runs `backend.worker infer` | bucket creds; a GPU host; a **net-new orchestration endpoint** |
REAL (remote) | FastAPI loop and worker CLI are **separate** today; "remote GPU as a setting" is
**design** ? selector shown **disabled** in MVP |
106:- **S1 Ingest & Process** ? the source menu ($4.2) + a segmenter picker from
`/api/capabilities` ? `POST /api/process`.
109:- **S4 Reels / History** ? needs `GET /api/videos` + `GET /api/reels/{video_id}` for
**reload-survival**.
113:**Reused (no re-plumb):** the full `/api` loop; the in-process worker; the chunked upload
endpoints (un-called); LocalStorage default; the detector seam incl. the landing
`NativeWasbRunner`; the **RallyPicker** design + anchors; the `web/` scaffold; the
`HOSTING_PLAN` gate; the `SETUP_NEW_MACHINE` runbook. **Net-new:** Caddyfile + systemd units;
`GET /api/capabilities`; `GET /api/videos`; `GET /api/reels/{video_id}`; `GET /api/health`; the
SPA console screens; ingest-from-bucket wiring; a retention sweep; (reserved) the bucket-worker
bridge.
135:| No Dockerfile | runbook-only reproducibility | feasibility-check a worker image before
promising Cloud Run/GCE |
140:Does **not** add multi-tenant identity/RLS (single-tenant alpha; uploads/outputs global).
Does **not** run real WASB on the CPU droplet (Gemini or stub only). Does **not** make the
remote-GPU bucket path work (design; selector disabled). Does **not** provide in-UI source
playback, job cancel, or a true progress bar (coarse free-text stage, poll-only). Does **not**
containerize (runbook; no Dockerfile `[verified]`). Does **not** verify native WASB (owner byte-
parity on a real GPU box, not this offline suite). A green SPA + contract tests prove the
**console plumbing**, not engine rally-detection quality (a separate track).
164:| T7 | *(engine)* `GET /api/capabilities` + `GET /api/health` + banner | T1 | coord | ? | ?
|
165:| T8 | *(engine)* `GET /api/videos` + `GET /api/reels/{video_id}` (S4 reload-survival) | ? |
coord | ? | ? |
184:- [ ] `/api/capabilities` + `/api/health` + banner truthful; reload-survival via
`/api/videos` + `/api/reels`.

```

16. user (2026-06-21T14:05:51.778Z)

```

Dockerfile
README.md
---DOCKERFILE HEAD---

```

Cloud Run GPU ****Job**** image for the rally worker (COMPUTE_DECOUPLED_SERVING)

Runs `python -m backend.worker infer` ? INLINE on an L4 (detect + window + stitch co
CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the own
M7 real run (the \$100/?10k budget, D17); CI does NOT build this (a CUDA image is too
dispatch shim is proven with a fake client (tests/test_compute_backend.py).

Two python stacks, deliberately separate (a framework conflict otherwise ? same split
native GPU box, deploy/digitalocean/gpu_setup.sh):

- * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
- * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
(the native runner shells out to \$WASB_PYTHON), so the app's torch never co-insta

? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance
commercial sharing. They are NOT baked into this image; stage them at runtime (a b

mount) and point \$WASB_WEIGHTS at them. Resolve WASB-C3 before any public, n

```
FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
```

```
ENV DEBIAN_FRONTEND=noninteractive \  
    PYTHONUNBUFFERED=1 \  
    PIP_NO_CACHE_DIR=1 \  
    CONDA_DIR=/opt/conda \  
    WASB_REPO_DIR=/opt/models/WASB-SBDT
```

ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB en

```
RUN apt-get update && apt-get install -y --no-install-recommends \  
    ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \  
    python3 python3-pip python3-venv \  
    && rm -rf /var/lib/apt/lists/*
```

--- WASB detector env (mirrors deploy/digitalocean/gpu_setup.sh ? the verified torch-

```
RUN curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o  
/tmp/mc.sh \  
    && bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh \  
    && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel  
https://repo.anaconda.com/pkg/main || true \  
    && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel  
https://repo.anaconda.com/pkg/r || true \  
    && "$CONDA_DIR/bin/conda" create -y -q -n wasb python=3.8 \  
    && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \  
    && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \  
        torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url  
https://download.pytorch.org/whl/cu113 \  
    && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python" -m pip  
install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \  
    && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \  
        hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml matplotlib  
"numpy<1.24"
```

--- Main application env ---

```
WORKDIR /app  
COPY pyproject.toml README.md ./  
COPY backend ./backend  
COPY training ./training  
RUN python3 -m pip install --upgrade pip && python3 -m pip install .
```

**The native runner finds the WASB env + repo by env (no `conda activate` needed); we
runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stay**

```
ENV WASB_PYTHON=/opt/conda/envs/wasb/bin/python \  
    WASB_REPO_DIR=/opt/models/WASB-SBDT \  
    WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \  
    RALLY_DETECTOR_IMPL=native
```

The dispatcher (CloudRunCompute) supplies the per-execution args:

infer --video-uri gs://? --out-uri gs://? --done-uri gs://? --segmenter wasb_hybrid --set

17. assistant (2026-06-21T14:05:56.063Z)

I now have strong grounding. The Dockerfile uses Miniconda with `conda tos accept` (the Anaconda ToS issue the digest flags ? micromamba/conda-forge is the clean path, NOT yet done). The bind is hardcoded `127.0.0.1`. Net-new endpoints confirmed absent. Let me check the worker train ship-gate, the i18n/SPA versioning, and the database migration ladder to ground the data-migration gap.

18. user (2026-06-21T14:05:58.161Z)

```
12: def __init__(self, db_path: str):
37:     conn.execute("PRAGMA foreign_keys = ON")
39:     conn.execute("PRAGMA busy_timeout = 30000")
41:     conn.execute("PRAGMA journal_mode = WAL")
69:     ``user_version`` stays un-bumped ? and the next open would re-run the same
88:     # Versioned migrations live here. PRAGMA user_version is the schema
91:     version = cursor.execute("PRAGMA user_version").fetchone()[0]
110:     # future: if version < 7: cursor.execute("ALTER TABLE ..."); PRAGMA user_version
= 7
183:     cursor.execute("PRAGMA user_version = 1")
210:     cursor.execute("PRAGMA user_version = 2")
222:     cursor.execute("PRAGMA user_version = 3")
242:     cursor.execute("PRAGMA user_version = 4")
280:     CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
290:     cursor.execute("PRAGMA user_version = 5")
337:     cursor.execute("PRAGMA user_version = 6")
=== TRAIN GATE ===
1: ""Gen-0 TAL harness (M0.4): ONE command -> train -> honest LOVO eval -> ship-gate ->
artifact.
17: (WASB C3 unresolved => commercial ship refused; explicit F1@tIoU target still
142: def gate_verdict(eval_res: dict, floor: Optional[dict]) -> dict:
143:     ""Pre-flight ship-gate.
150:     Beating floor + sign test necessary; C3 main commercial blocker.
151:     ship=True only when all non-C3 blocks clear (honesty first).
174:     reasons.append("ship-gate F1@tIoU target NOT YET CALIBRATED ? provisional; see issue
#172")
175:     reasons.append("WASB checkpoint provenance C3 UNRESOLVED ? commercial ship blocked
regardless of F1")
176:     # ship remains False until C3 resolved + other gates (by design).
177:     return {"ship": False, "floor_passed": floor_passed, "sign_test": sign, "blockers":
reasons}
196:     verdict = gate_verdict(eval_res, floor)
235:     "wasb_checkpoint_provenance": "C3-UNRESOLVED (ship-blocking; see
COMMERCIALIZATION C3)",
236:     "no_paid_llm_training_labels": True,
248:     print(f"GATE : ship={verdict['ship']} floor_passed={verdict['floor_passed']} ")
```

19. user (2026-06-21T14:06:00.607Z)

```
7: const BASE = "/api";
=== SPA version sync ===
README.md
dist
index.html
node_modules
package-lock.json
package.json
src
tsconfig.json
vite.config.ts
---
4: "version": "0.0.0",
```

20. assistant (2026-06-21T14:06:07.574Z)

I have everything I need. The DB migration ladder exists (user_version 1-6) but there's no app-data-dir migration story; the ship gate hardcodes `ship=False`; SPA version is `0.0.0` with no engine-version contract. Let me do two final checks: the `/api/config` exposure (secrets in API responses?) and whether `.env`/GEMINI_API_KEY handling is anywhere.

21. user (2026-06-21T14:06:08.931Z)

```

@app.get("/api/config")
def get_config():
    return global_config

@app.get("/api/health")
def health():
    """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
    (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
    confirm the engine is reachable and which detector is wired. Never raises."""
    sport = getattr(global_config, "sport", None)
    indexing = getattr(global_config, "indexing", None)
    return {
        "status": "ok",
        "sport": sport,
        "default_segmenter": getattr(indexing, "default_segmenter", None),
    }

def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
    """The full hash ? validate ? segment ? report pipeline for one video.

    This is the former synchronous /api/process body, unchanged in behavior, now run on
    the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
    """
    env/key handling ===
backend/main.py:8:from dotenv import load_dotenv
backend/main.py:10:load_dotenv()

```

22. assistant (2026-06-21T14:06:15.165Z)

`GET /api/config` returns the entire `global\_config` object with no auth and no field redaction. `load\_dotenv()` is called at import with no path argument (CWD-relative). I have enough grounding to deliver a harsh, specific critique.

Key confirmed facts: net-new endpoints (`/api/capabilities`, `/api/videos`, `/api/reels`, segment-correction PATCH) are absent; `uvicorn.run(host="127.0.0.1")` is hardcoded; the Cloudrun Dockerfile uses Miniconda + `conda tos accept` (not the recommended micromamba/conda-forge); ship gate hardcodes `ship=False`; SPA is `version 0.0.0` with `const BASE="/api"` and no engine-version contract; DB migration ladder exists at user_version 6 but no app-data relocation/migration story; `GET /api/config` returns the whole config object unauthenticated; `load\_dotenv()` is CWD-relative.

23. user (2026-06-21T14:08:15.004Z)

Structured output provided successfully

24. assistant (2026-06-21T14:08:20.616Z)

I've completed the completeness critique and returned it via StructuredOutput. The findings are grounded in actual file reads: the CWD-relative state handling (main.py:10,71-72; loader.py:32; doctor.py:54), the hardcoded `uvicorn.run(host="127.0.0.1")` at main.py:756, the unauthenticated full-config dump at `GET /api/config` (main.py:144-146), the absent net-new endpoints (`/api/capabilities`, `/api/videos`, `/api/reels`, segment-correction PATCH), the WASB-C3 ship-block hardcoded in harness.py:177 and the Dockerfile, the Miniconda+`conda tos accept` still in deploy/cloudrun/Dockerfile, the DB user_version ladder with no migration/backup story, and the SPA's `0.0.0` version + hardcoded `BASE="/api"` with no version handshake.